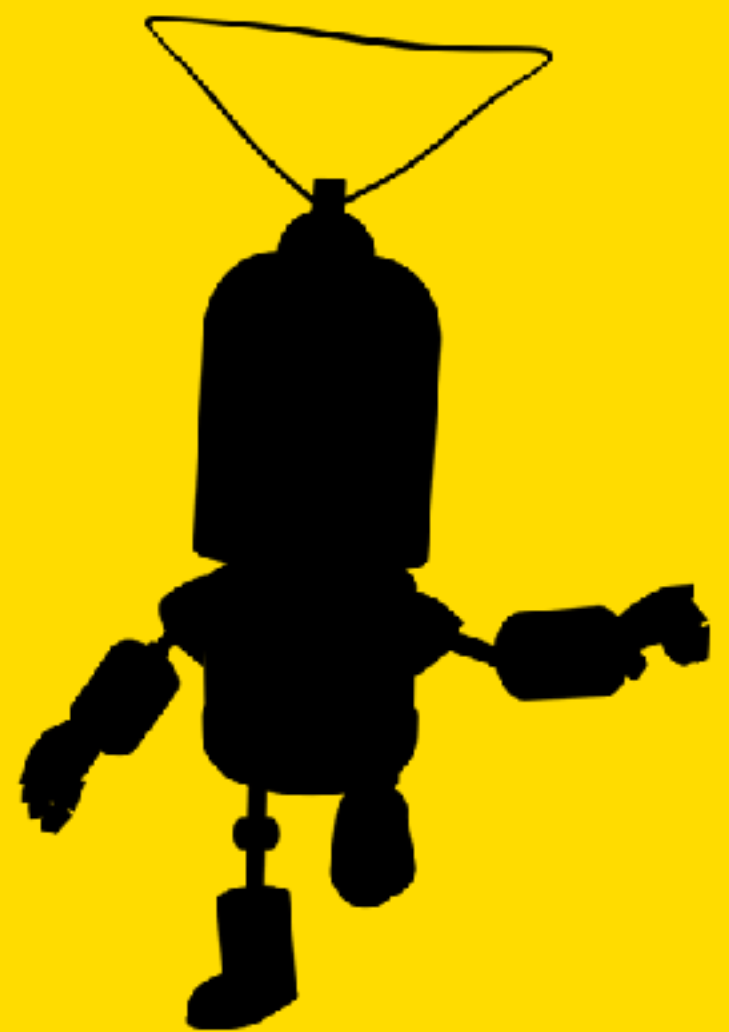


Agents & search



Oliver Obst



Overview

1	5/6	Introduction. Agents & search		
2	12/6	Planning		release assignment #1
3	19/6	Knowledge representation		
4	26/6	Neural networks		
5	3/7	Reinforcement learning		deadline assignment #1 (6 Jul)
6	10/7			release assignment #2
7	17/7	Reasoning with uncertainty		
8	24/7	Computer vision		
9	31/7	Foundation models		deadline assignment #2 (31 Jul)
10	7/8	Human-aligned intelligent robotics		

Week 1 content

- intelligent agents as goal-directed systems
- problem formulation: state, action, goal, cost
- state spaces and search trees
- uninformed search: DFS, BFS, uniform-cost
- heuristic search: intuition and A*
- example on a small office navigation map
- bridge to planning next week

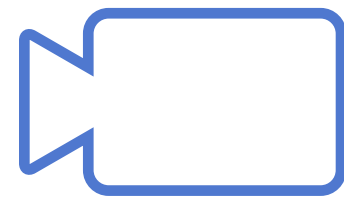
From AI to agent

Slides material and acknowledgement:

These slides contain material from

- Artificial Intelligence: A modern approach, by Peter Norvig and Stuart Russell
- Introduction to Artificial Intelligence, by Nikhil Sharma, Josh Hug, Jacky Liang, and Henry Zhu and the slides by Dan Klein and Stuart Russell
- the ai9414 python package

What's an agent?



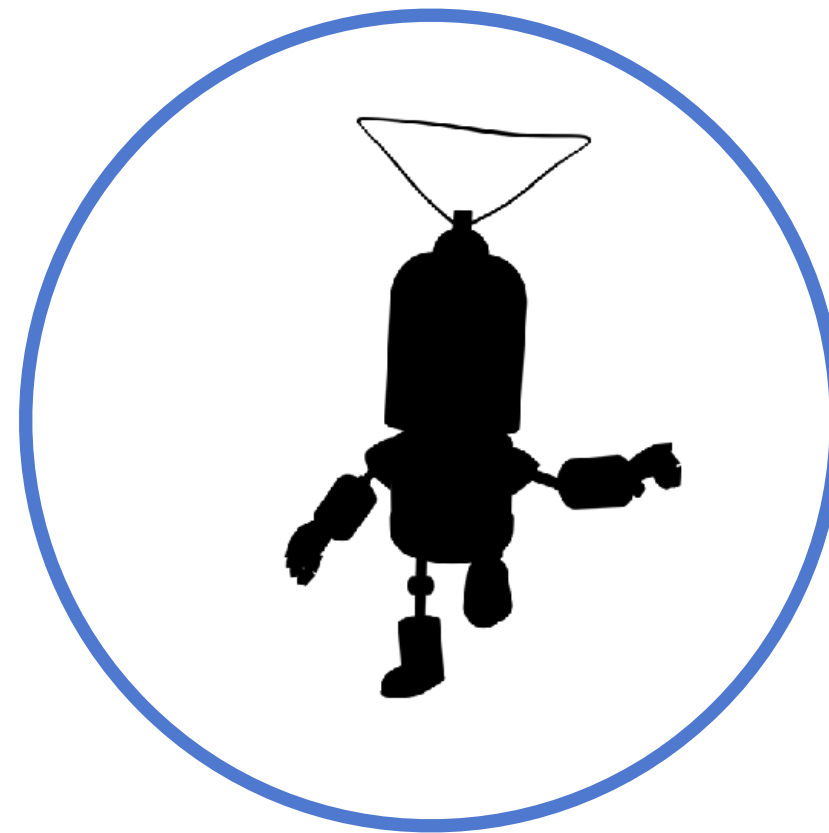
Sense



Think



Act

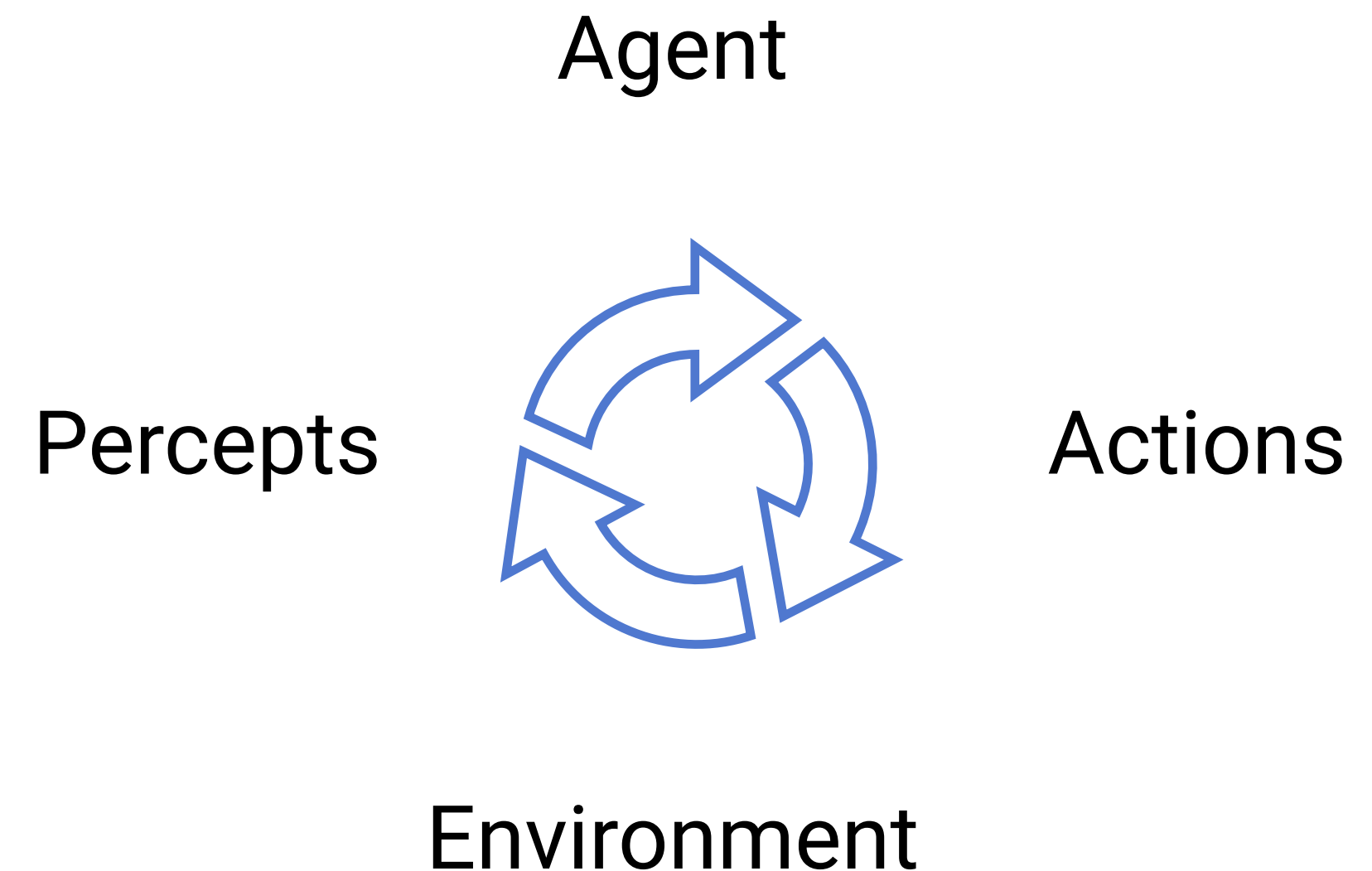


An agent is an entity that perceives, decides, and acts in an environment.

Environments

The environment is the external world in which an agent operates.

It provides percepts to the agent and changes in response to the agent's actions, and sometimes independently of them.



The environment is not just the map

Includes:

- Physical world
- Relevant objects
- Other agents or people
- Anything that could change a task or outcome



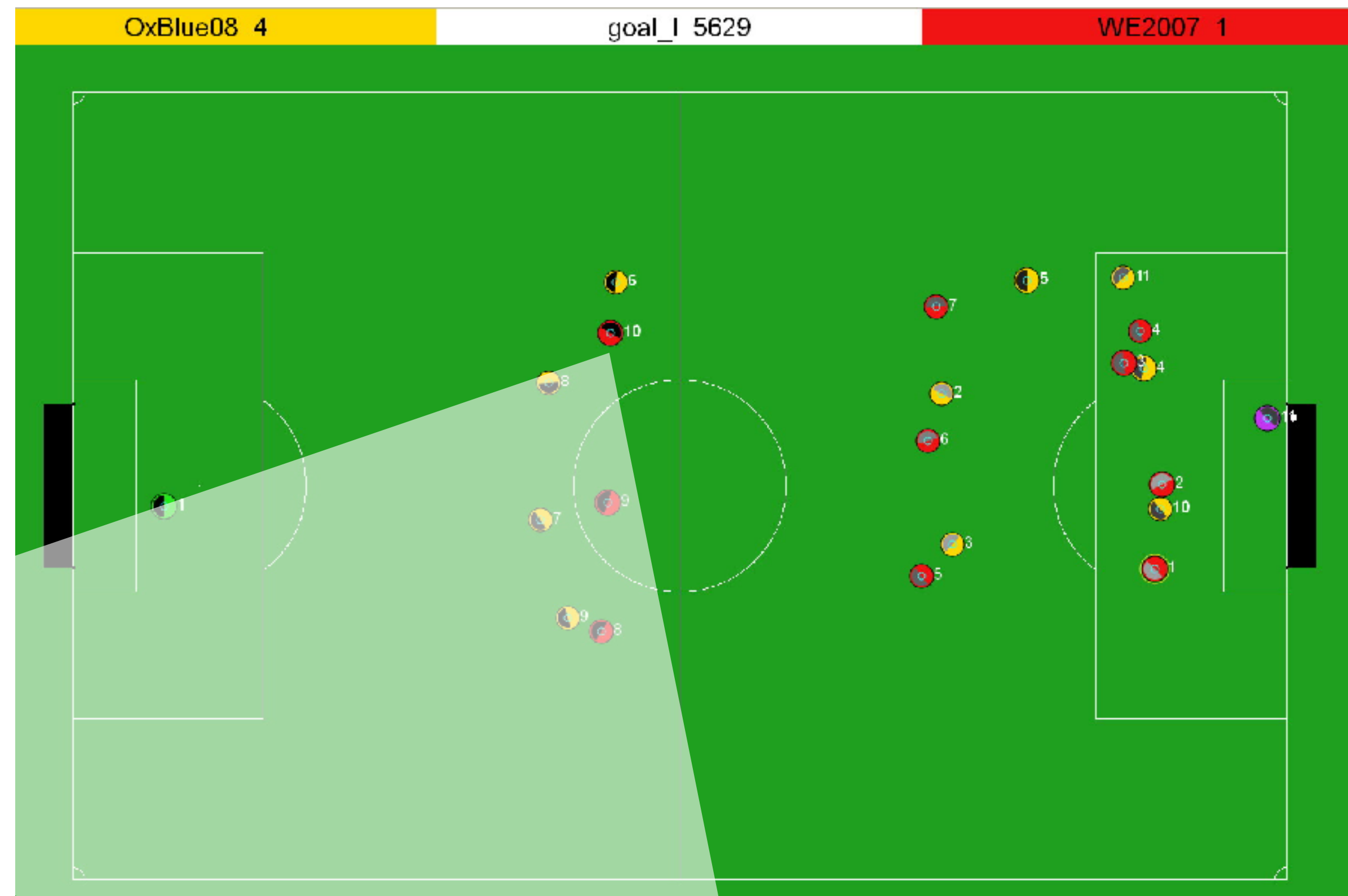
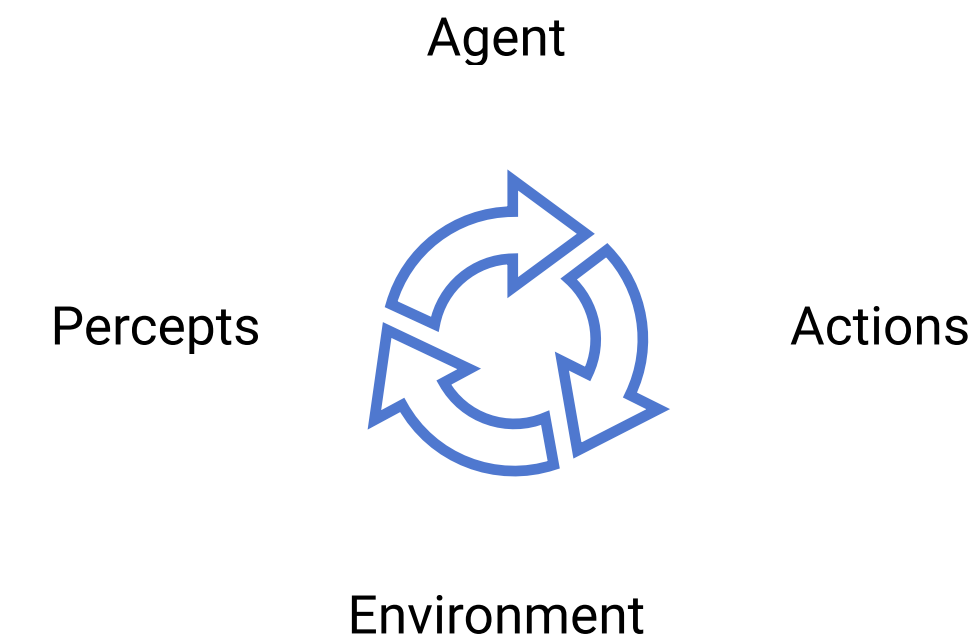
Picture <https://www.booster.tech/>

Perception

The agent usually does not see the whole environment directly

Environments can differ in important ways:

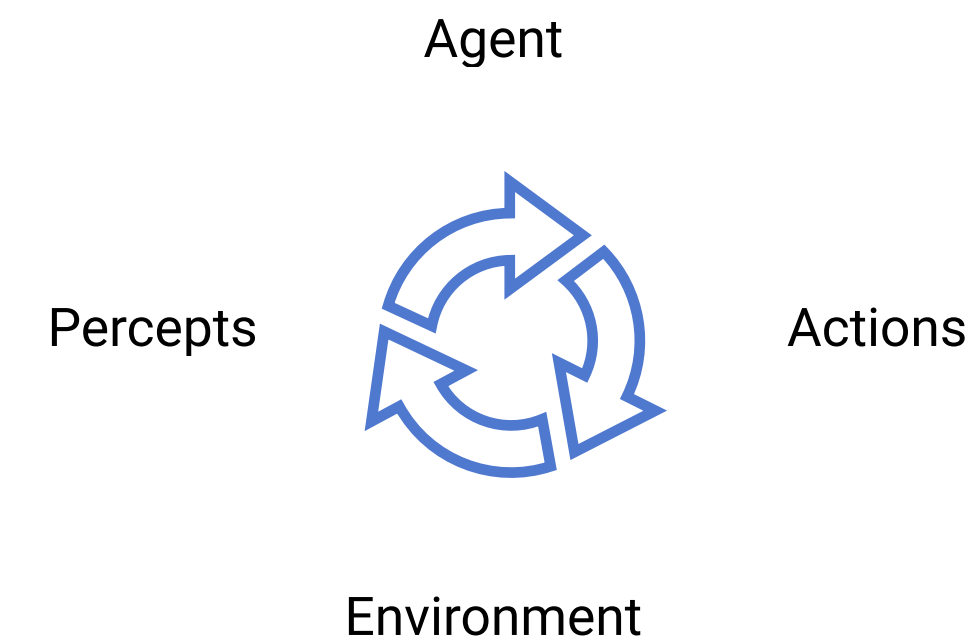
- fully observable vs partially observable
- deterministic vs stochastic
- static vs dynamic
- discrete vs continuous



Running example: delivery robot

- agent: delivery robot controller
- environment:
office building with rooms, doors, parcels, people
- percepts: location estimate, camera, door status
- actions: move, pick up, deliver

Good AI starts with a good model of the environment.



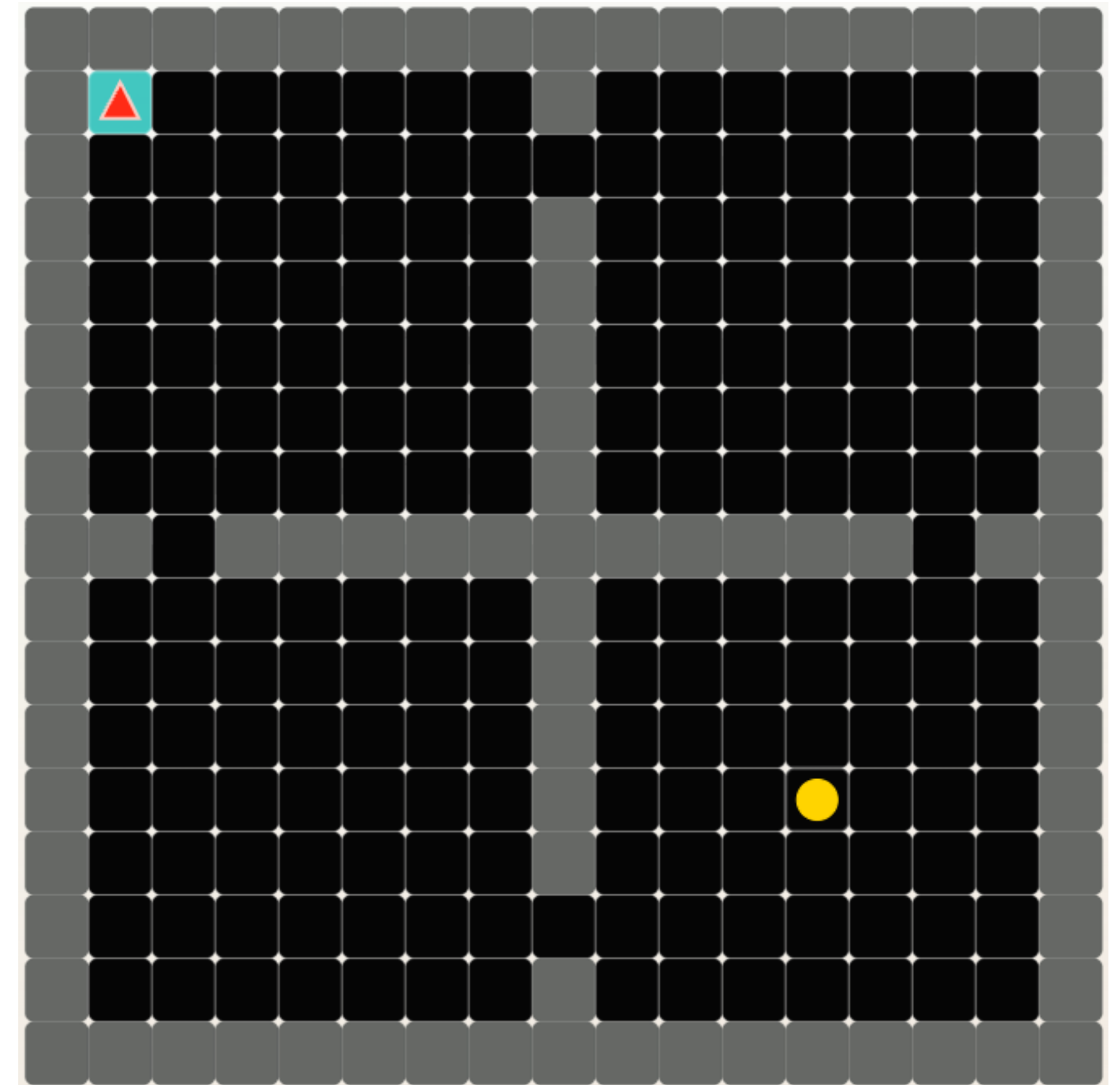
Running example

We start with the simplest version of the problem:
navigation to a goal.

The robot ▲ is in the mail room.

The goal is to navigate to the lab at ●.

Later weeks will make the same environment
more difficult; with parcels, keys, doors,
uncertainty, and so on.



What makes something a search problem?

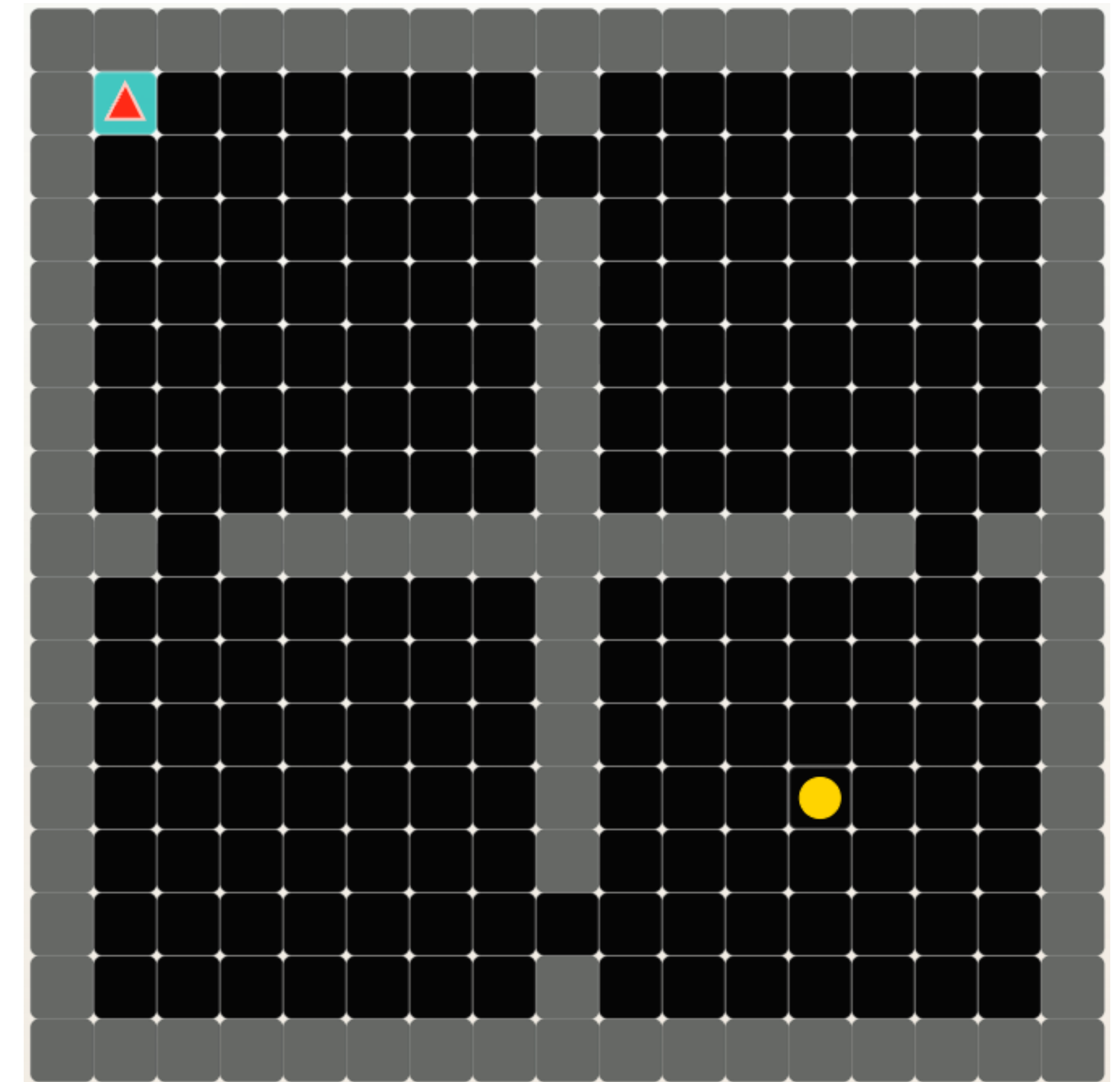
We start with the simplest version of the problem: navigation to a goal

We need to define the “ingredients” to our problem:

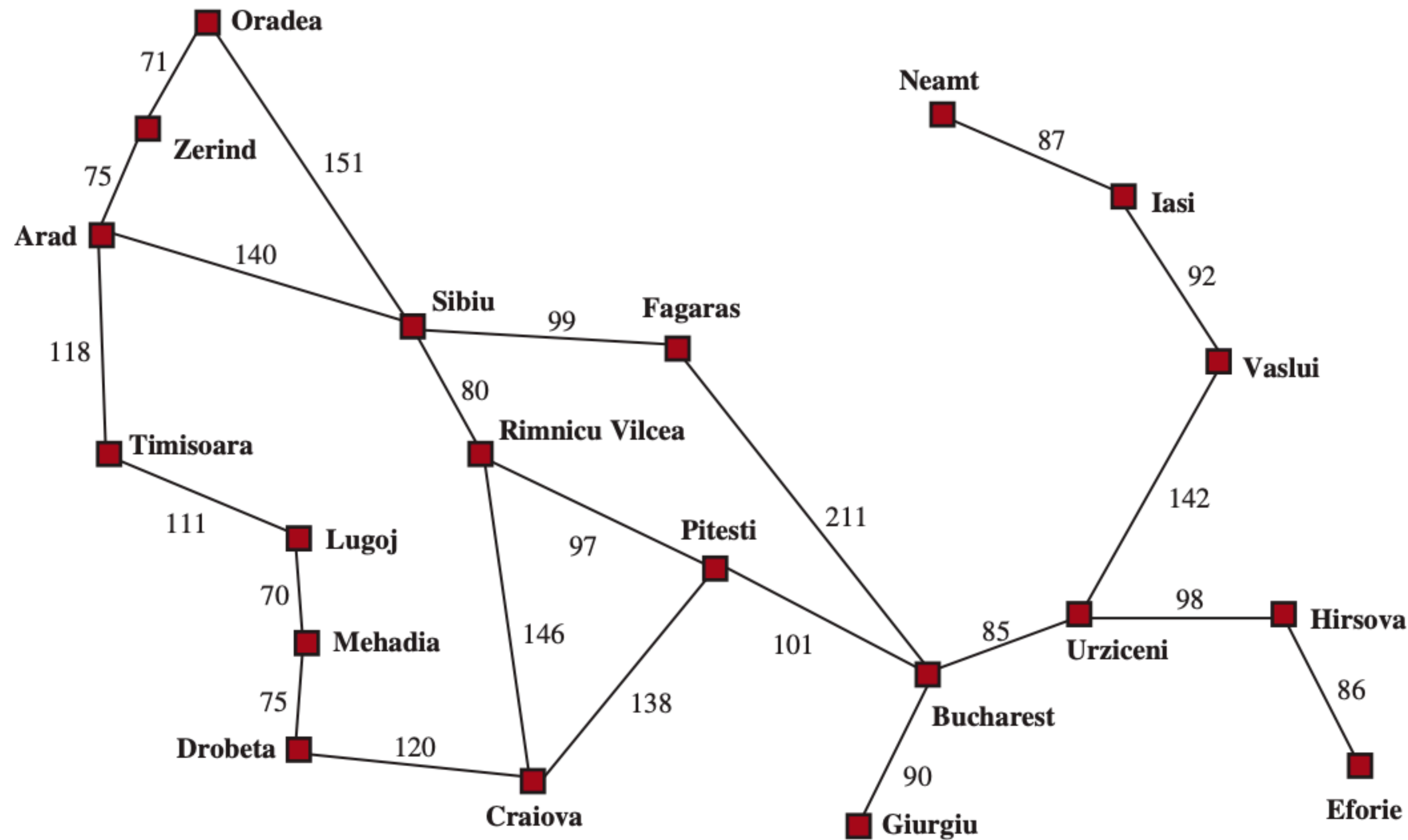
- Initial and current state
- Actions available / successor function
- Goal test: are we there yet?
- Path cost

A **rational agent** selects actions that maximise utility.

The representation often matters as much as the search algorithm.
It determines what actions the agent can even consider.



Problem formulation example



State space:

- Cities

Successor function:

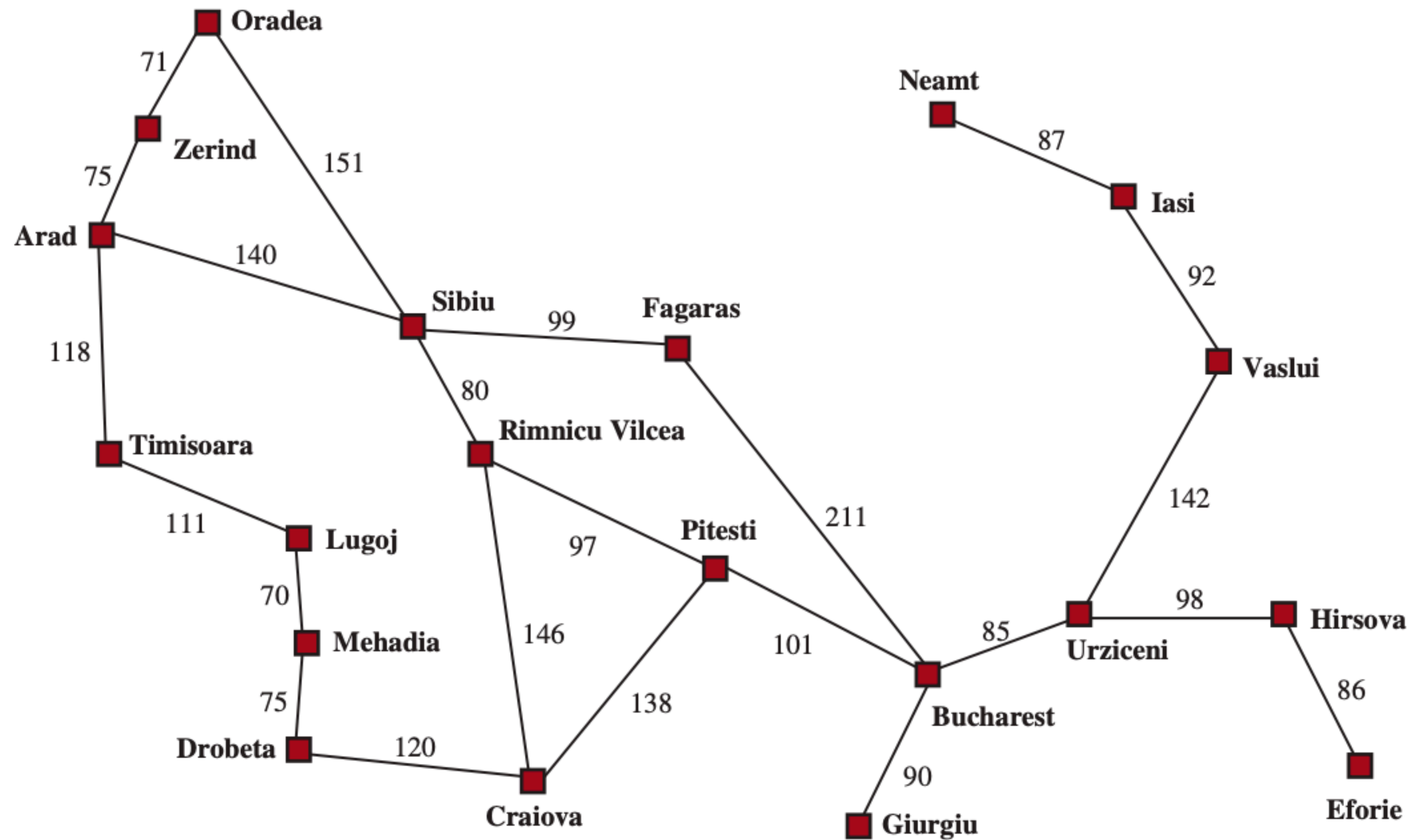
- Road: go to adjacent city with cost = distance

Start state: Arad

Goal test:

- Is state == Bucharest yet?

Problem formulation example



State space:

- Cities

Successor function:

- Road: go to adjacent city
with cost = distance

Start state: Arad

Goal test:

- Is state == Bucharest yet?

State spaces

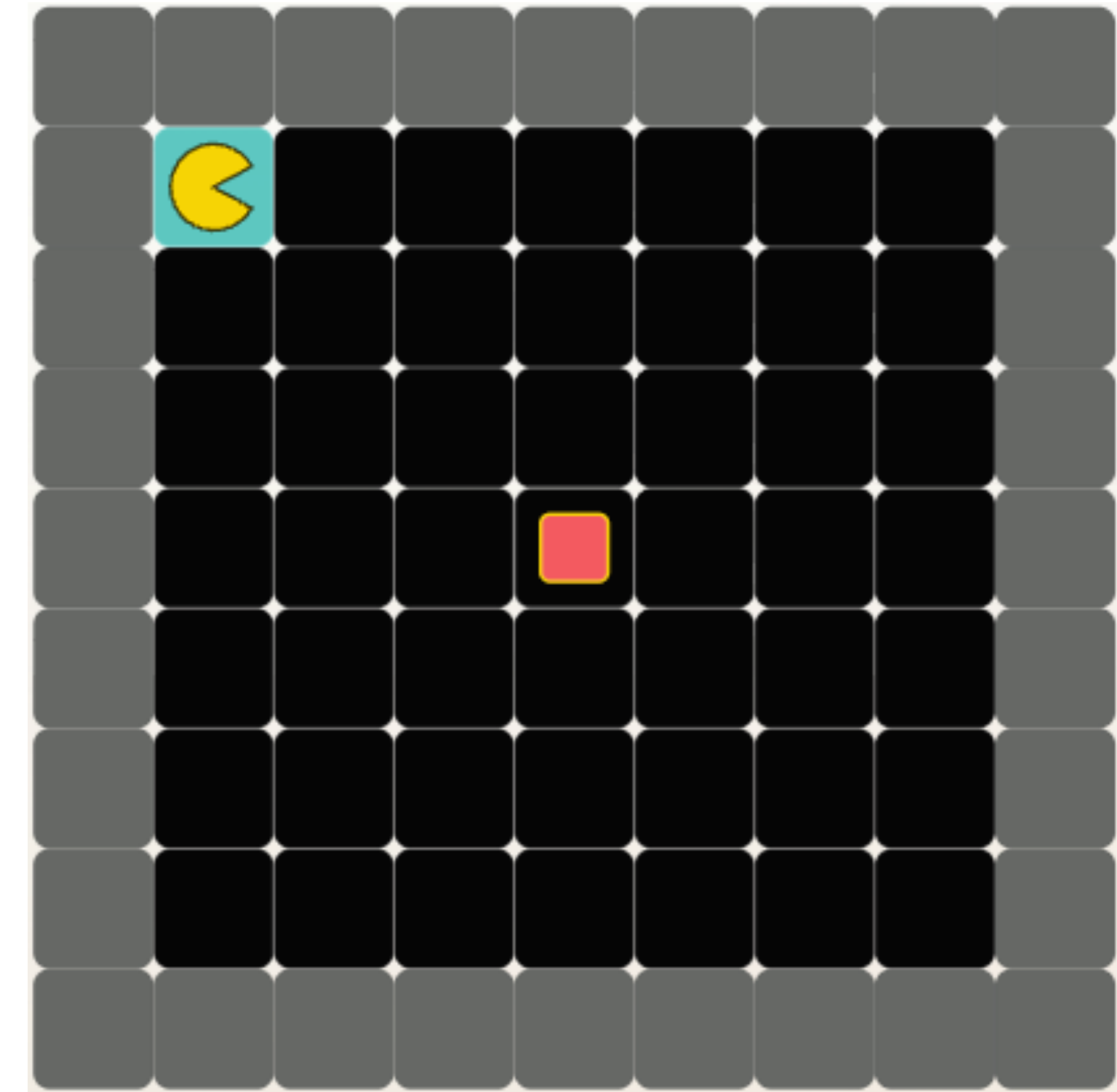
World state

- Agent position on a 7x7 grid
- Agent direction (NESW)
- Goal position on a 7x7 grid

How many are there:

- World states: $49 \times 4 \times 49$
- for finding a specific path: 49×4 - the goal is fixed

The representation decides what the search has to remember.



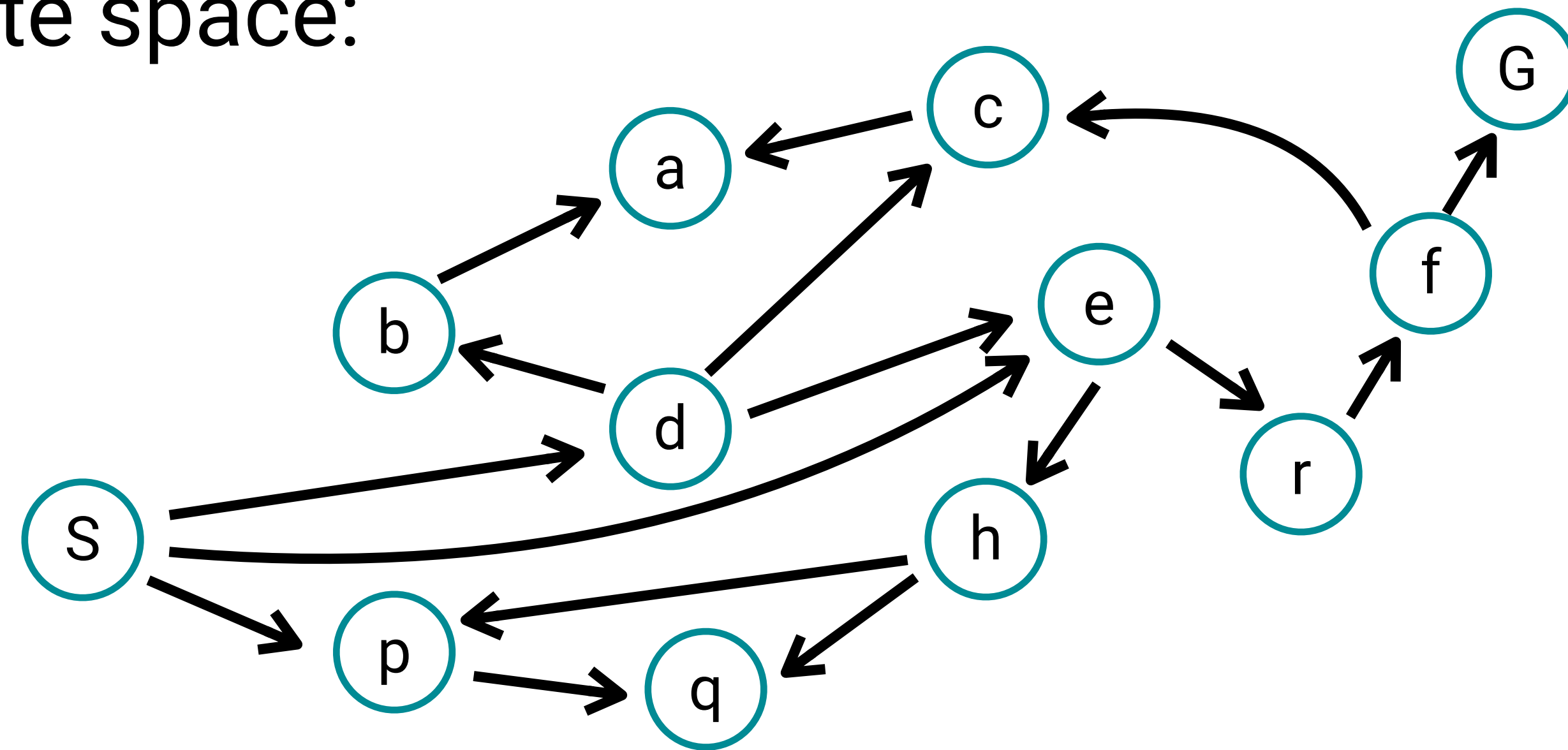
State space graphs

We can represent the problem in state space:

- each world state is a node
- edges represent successors

Each world state occurs only once.
Multiple ways to get there.

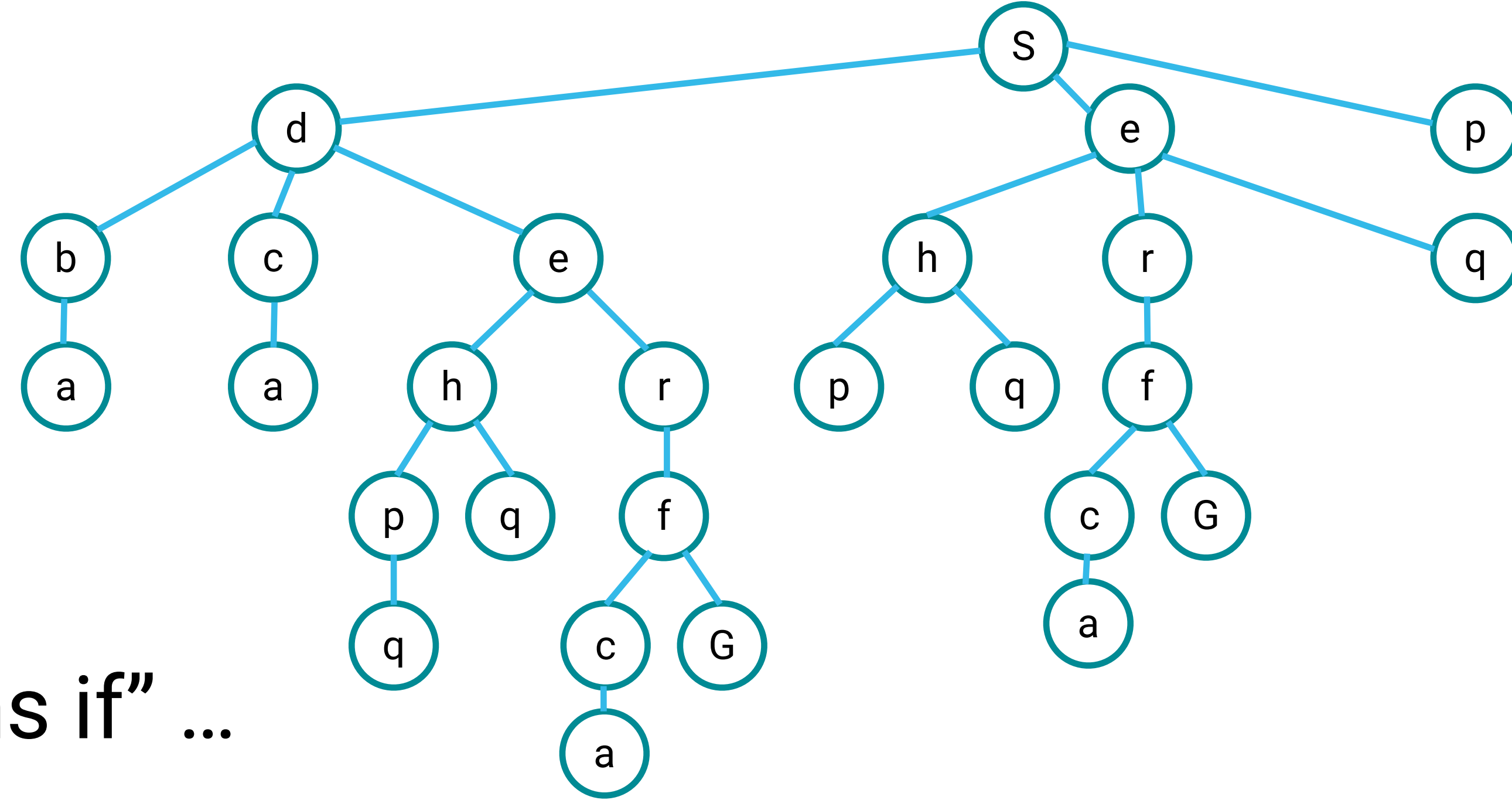
- we may have loops.
- graph may become very big



Search trees

We can also represent the problem as a search tree

Think of it as a tree of “what happens if” ...

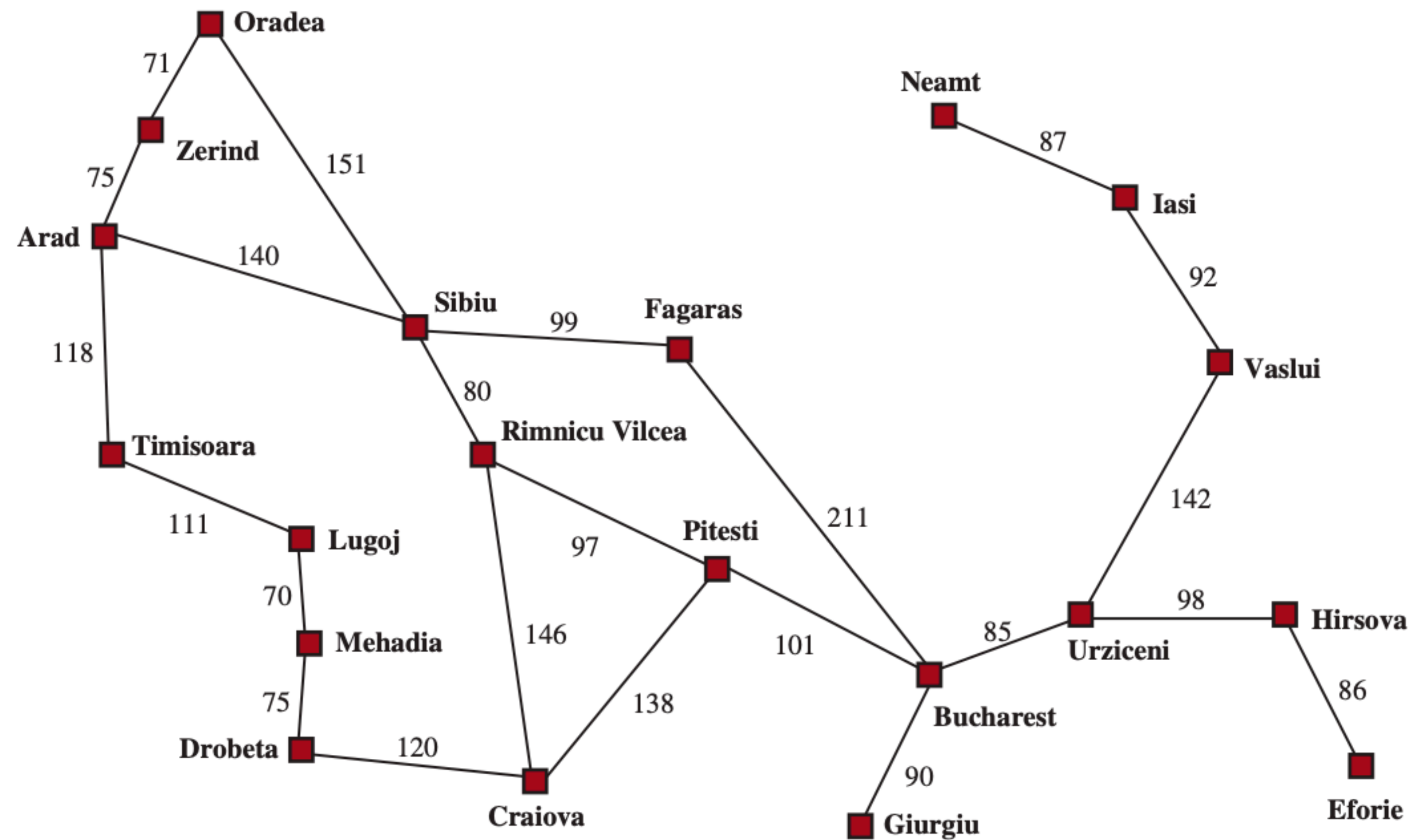


Children of a node are successors.

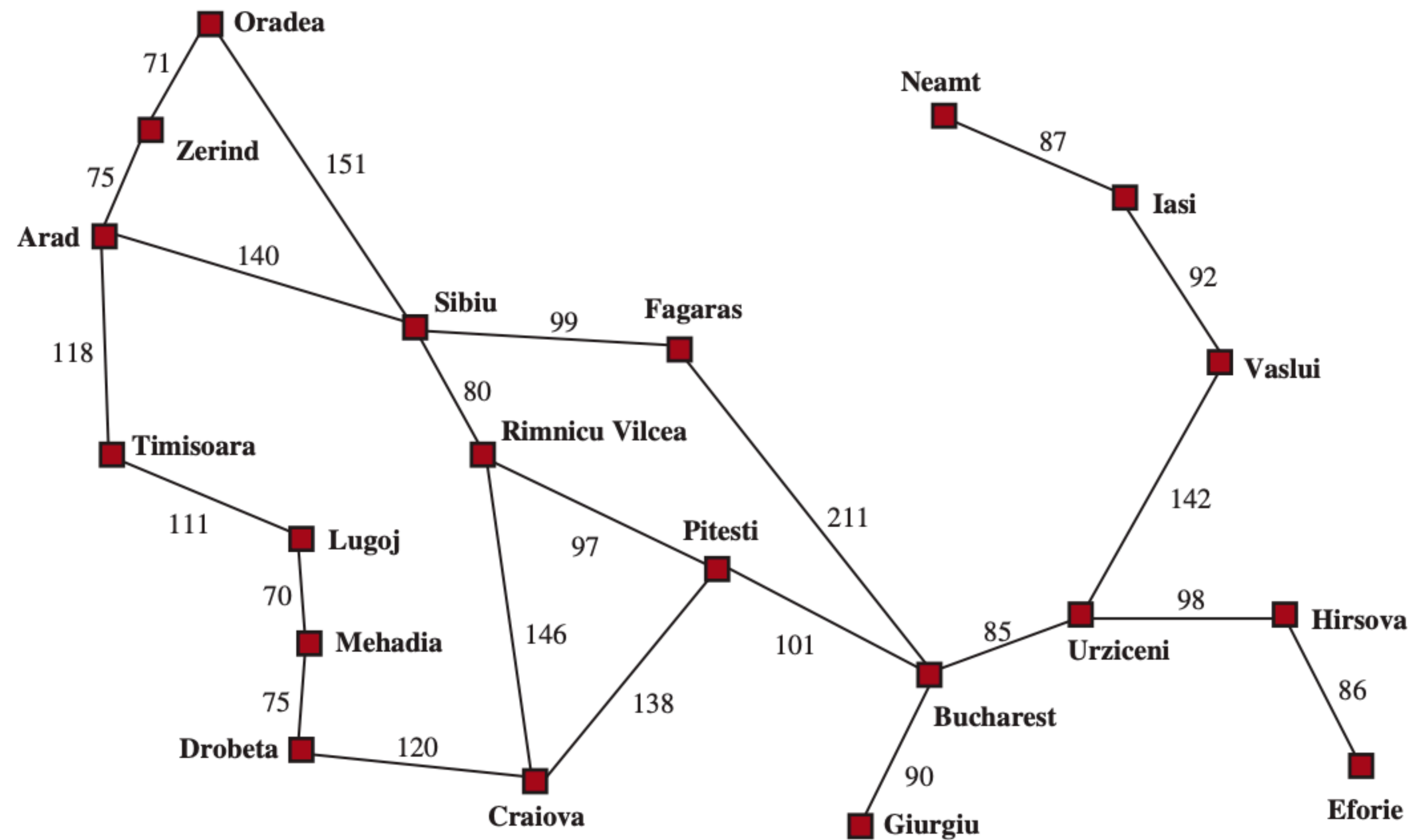
A node is a state, corresponding to a plan how to get there.

Tree for most problems infinite. So we never build the whole tree.

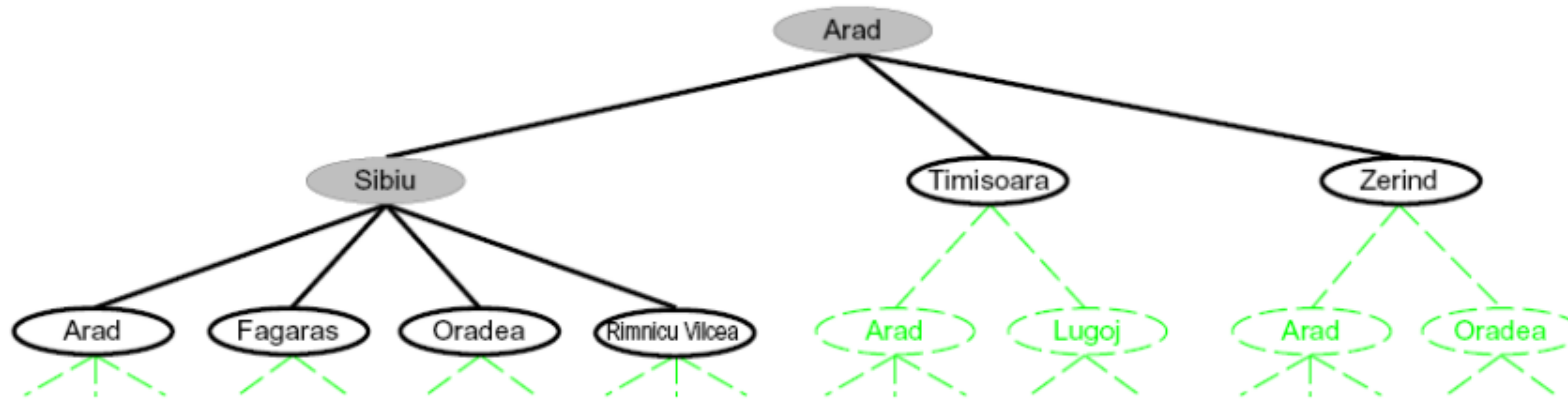
Romania revisited



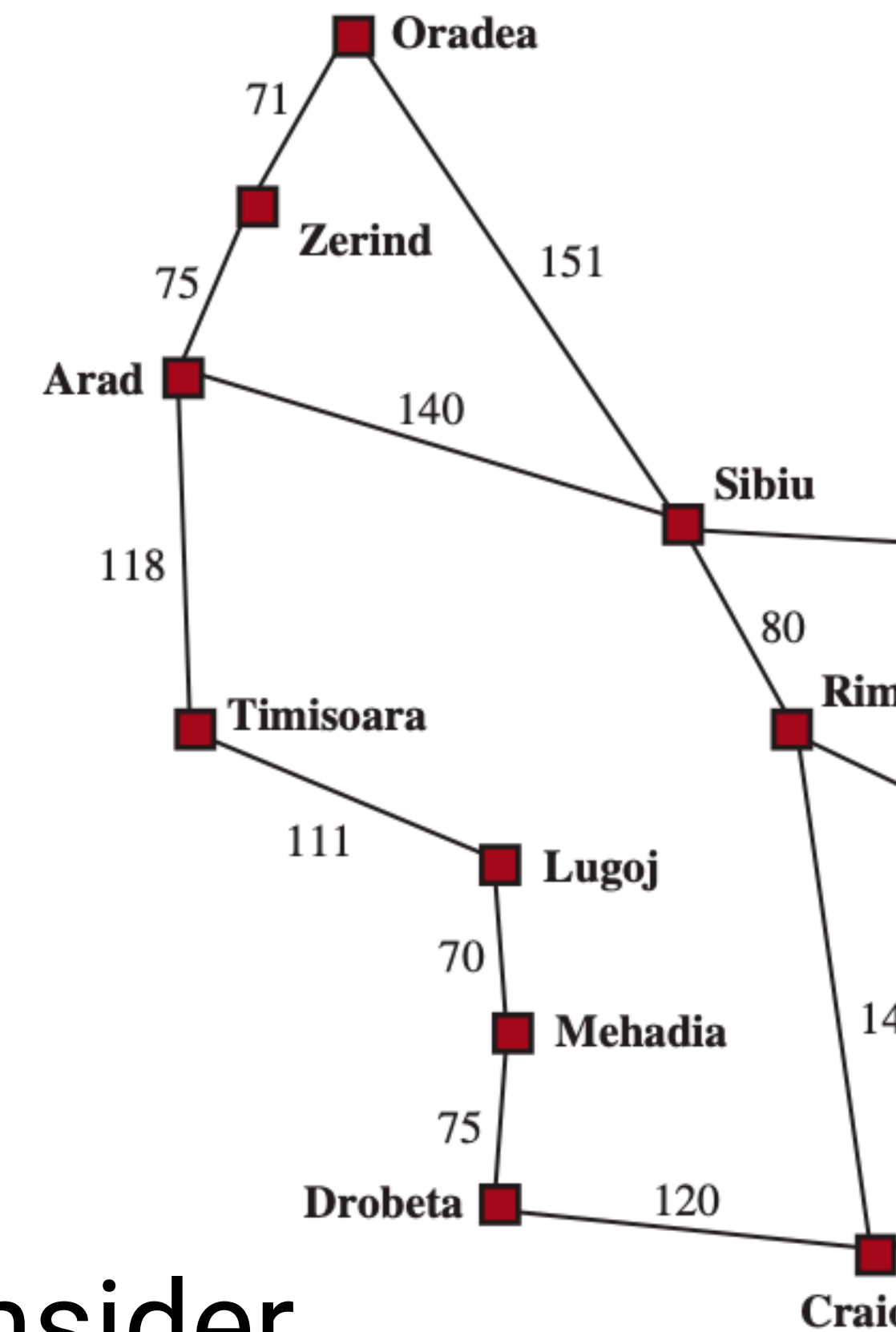
Romania revisited



Searching with a search tree



- Expand potential plans (new nodes in tree)
- Maintain a frontier (fringe) of partial plans we still consider
- Expand as few tree nodes as possible



General tree search

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

Frontier (fringe)/ Expansion / Exploration strategy

Main question: Which nodes on the frontier to explore next?

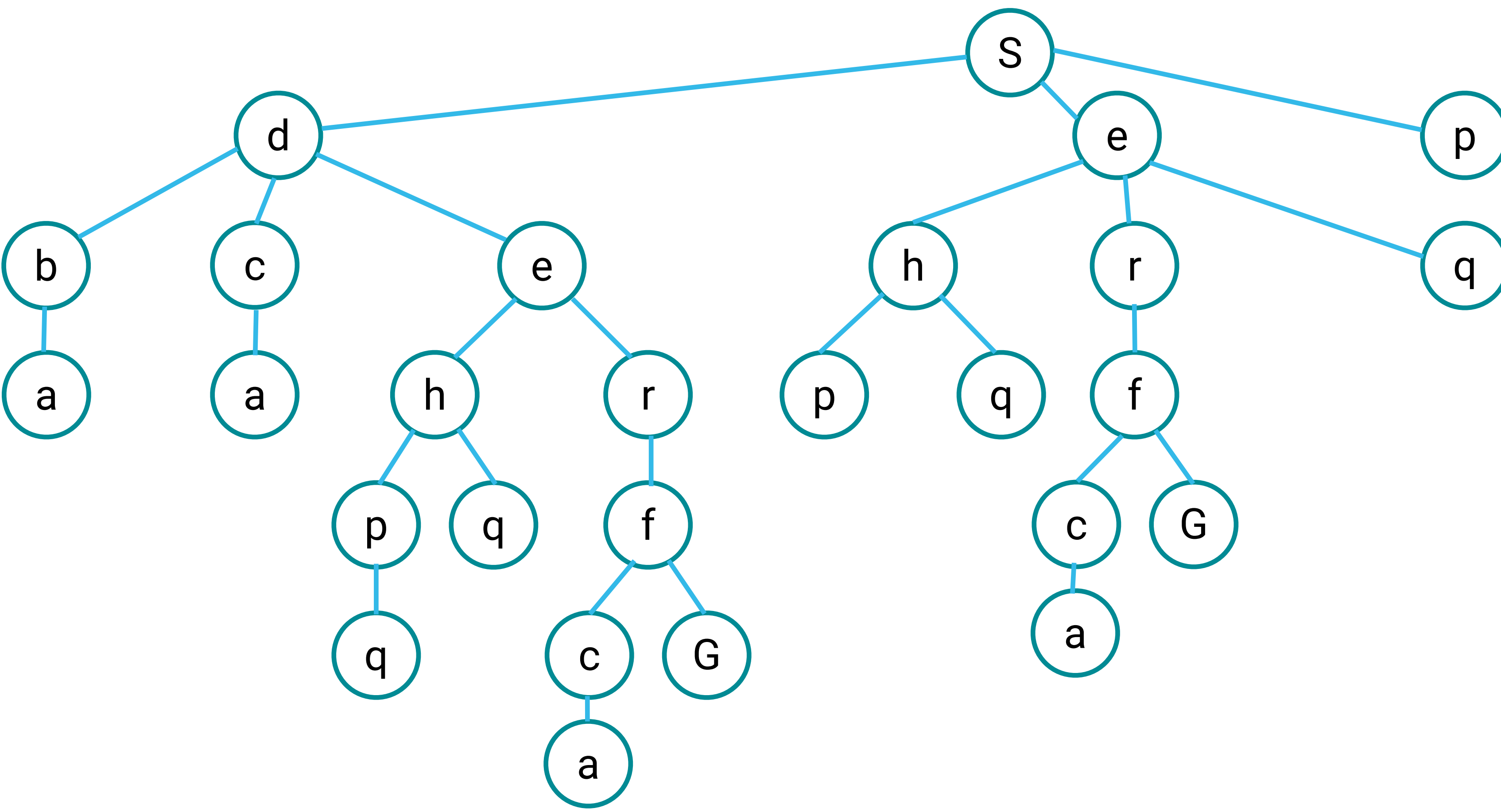
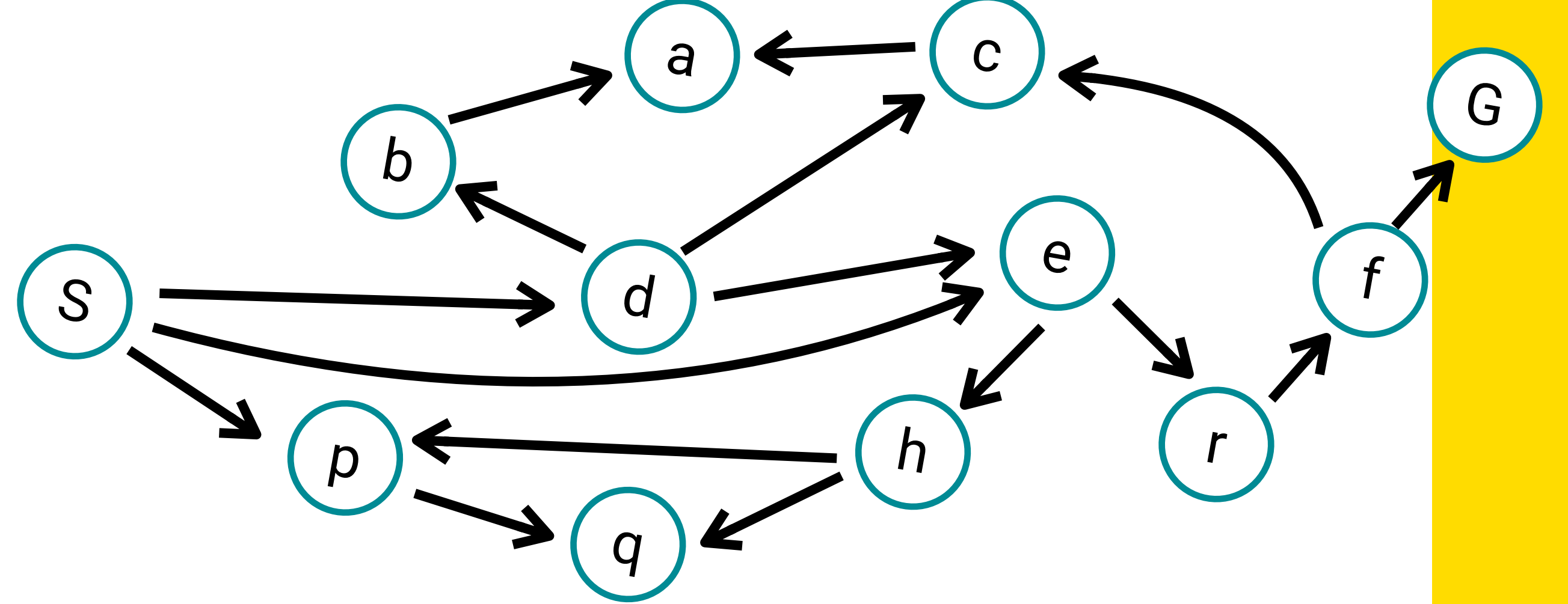
General tree search

Main question: Which nodes on the frontier to explore next?

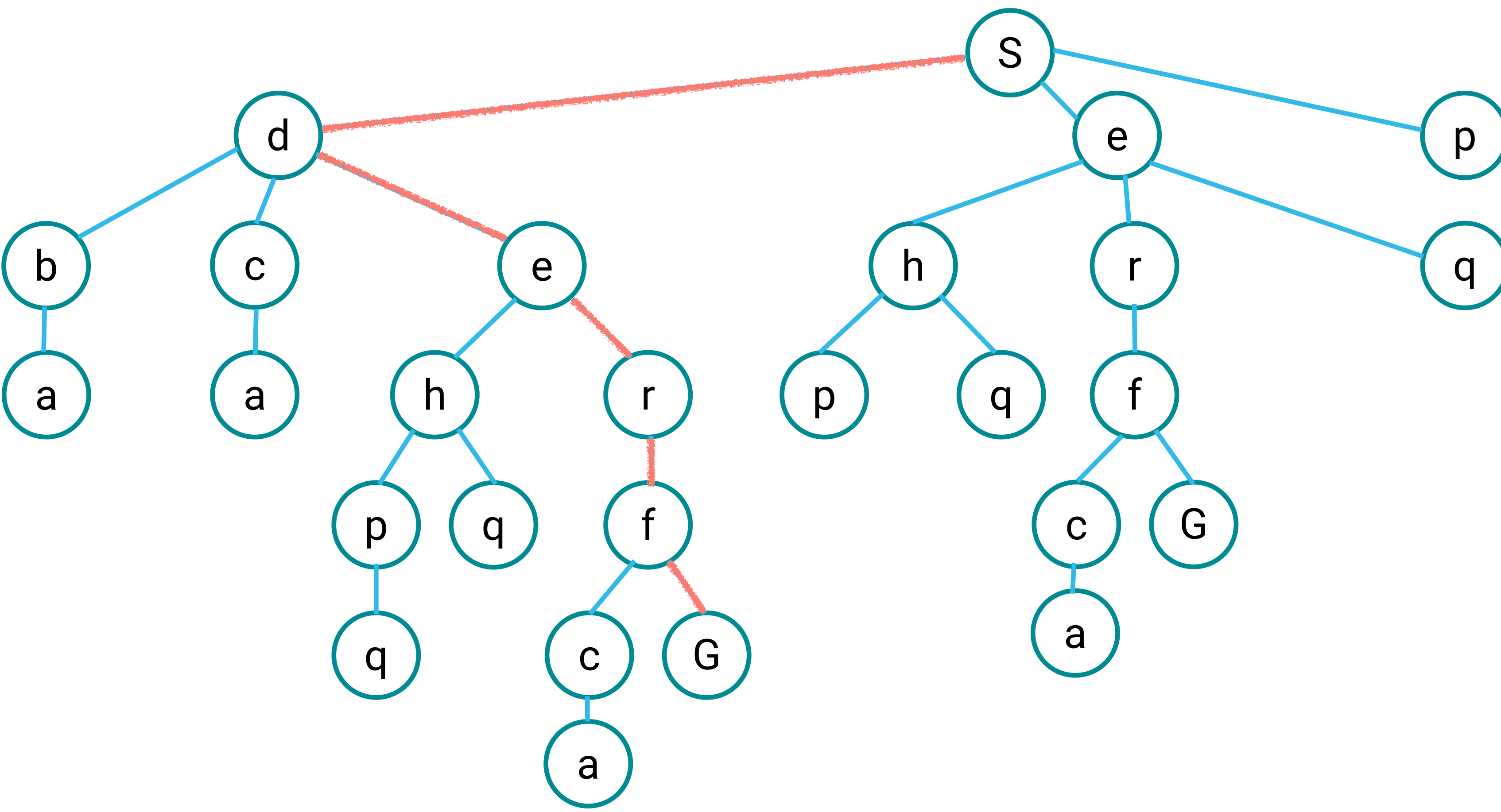
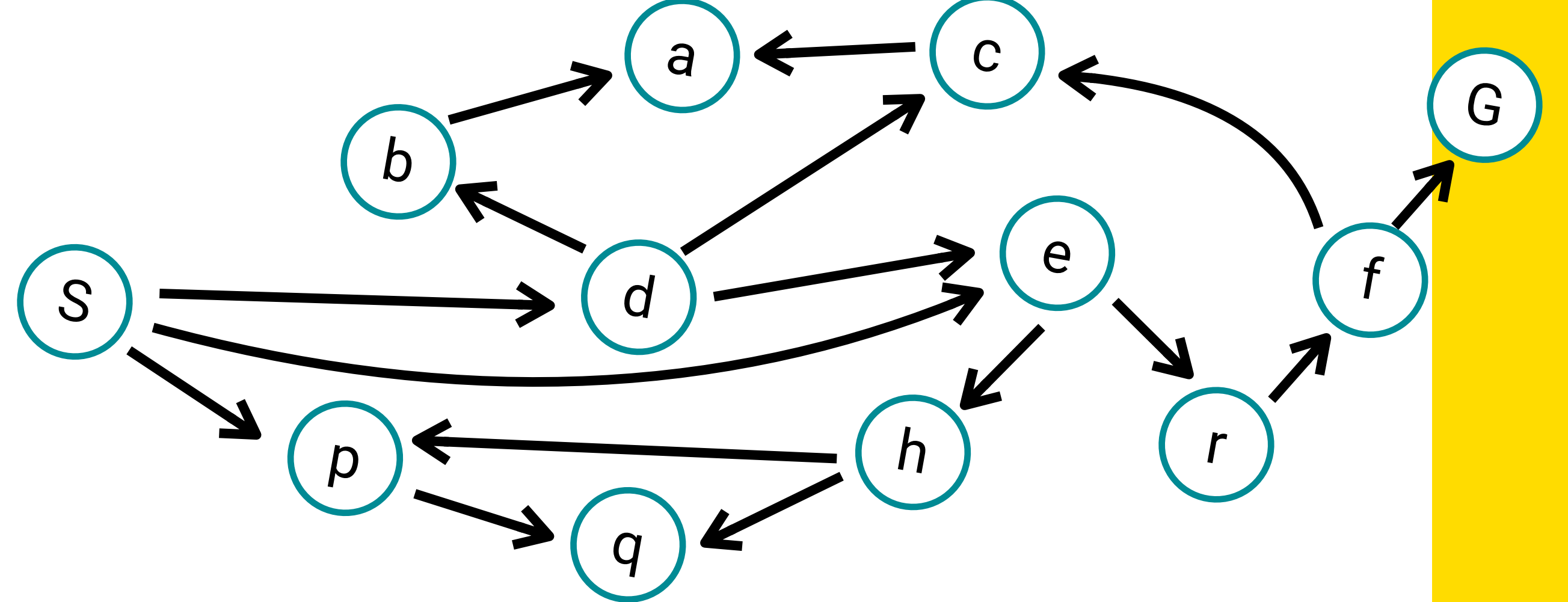
Frontier → Choose node → Goal test

→ Expand → add successors → Frontier

Example tree search



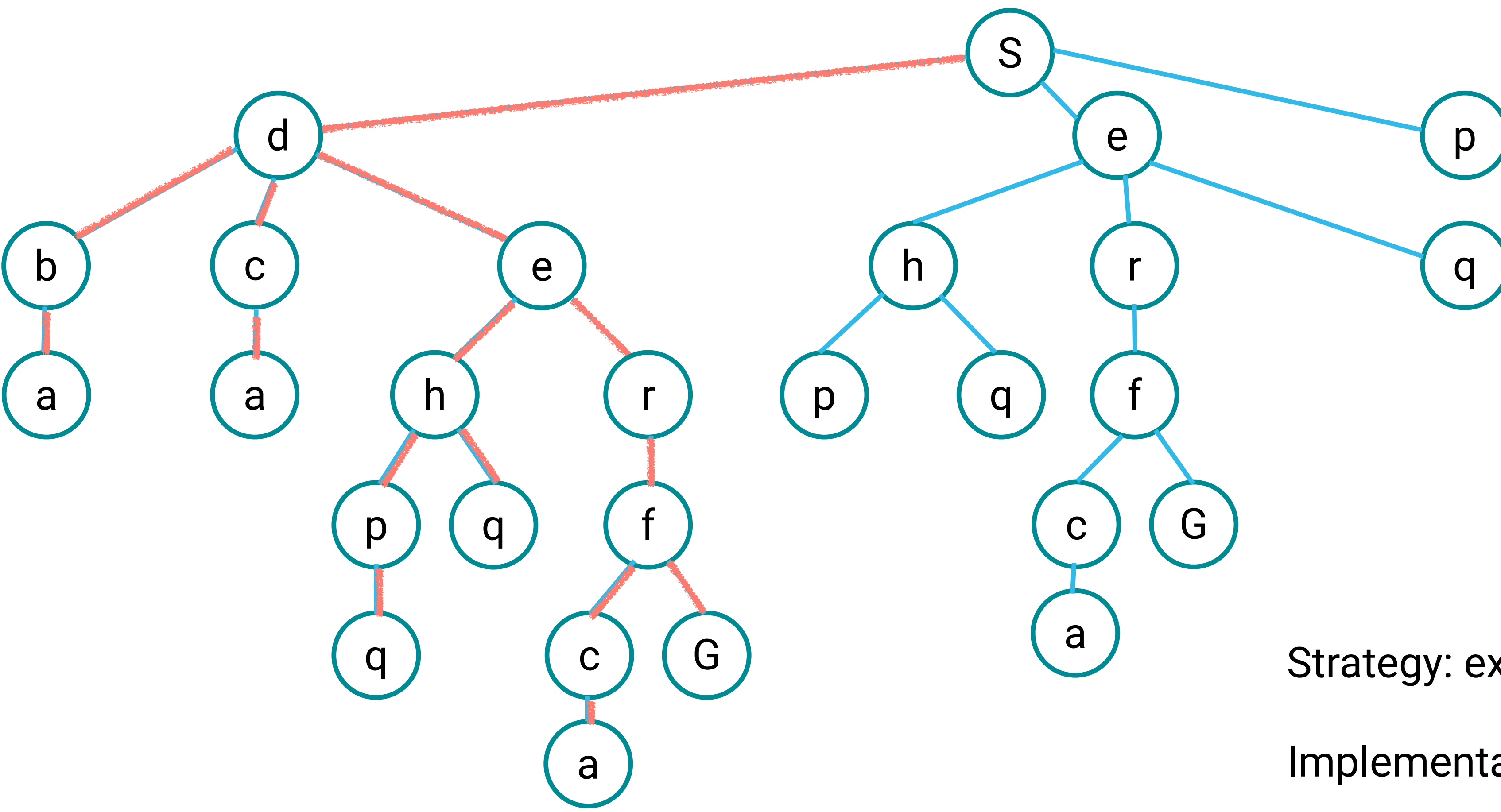
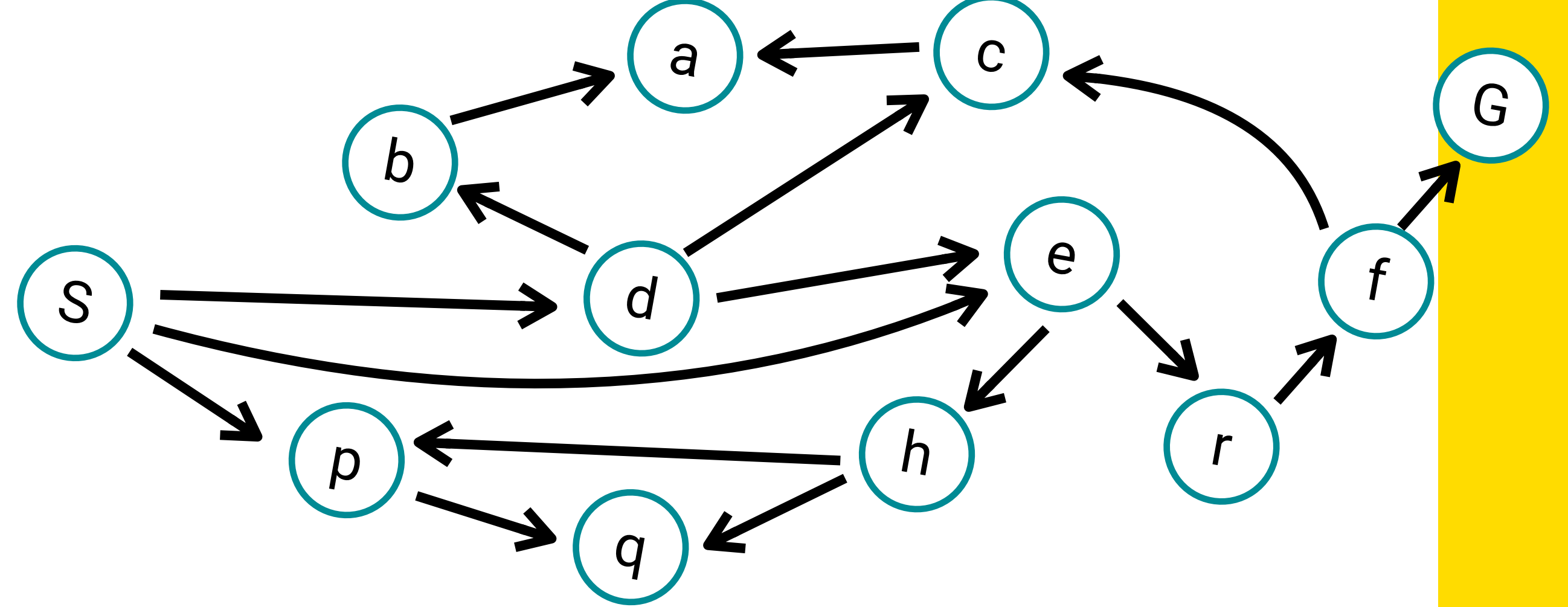
Example tree search



~~S~~
~~S → d~~
S → e
S → p
S → d → b
S → d → c
~~S → d → e~~
S → d → e → h
~~S → d → e → r~~
~~S → d → e → r → f~~
S → d → e → r → f → c
~~S → d → e → r → f → G~~

Depth first search

Depth first search



Strategy: expand a deepest node first

Implementation: Frontier is a LIFO stack

Properties of search algorithms

Completeness: Guaranteed to find a solution if one exists?

Optimality: Guaranteed to find the least cost path?

Time complexity?

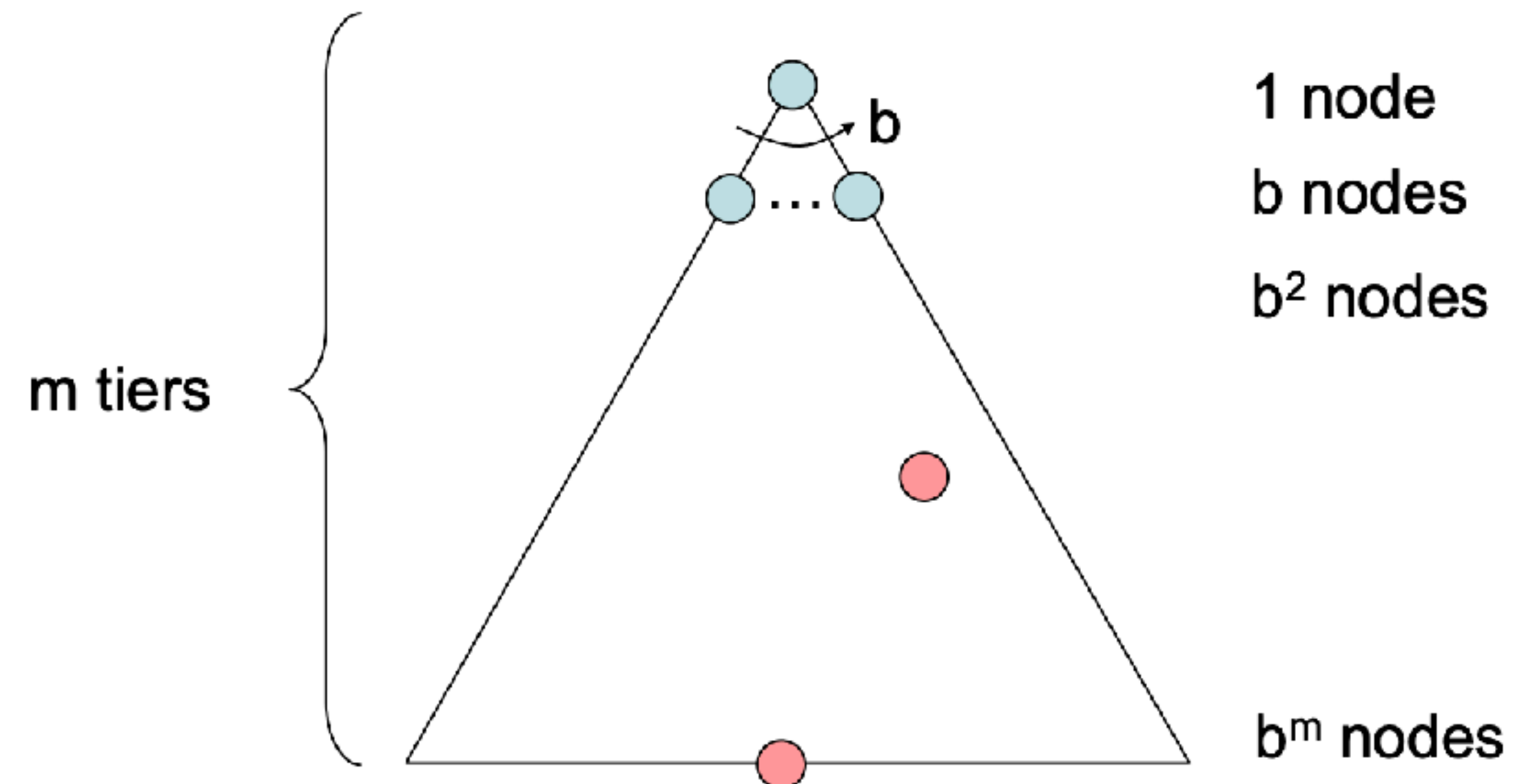
Space complexity?

Cartoon of search tree:

- b is the branching factor
- m is the maximum depth
- solutions at various depths

Number of nodes in entire tree?

$$1 + b + b^2 + \dots + b^m = \mathcal{O}(b^m)$$



Properties of depth first search

What nodes does DFS expand?

- Some left prefix of the tree.
- Could process the whole tree!
- If m is finite, takes time $\mathcal{O}(bm)$

How much space does the fringe take?

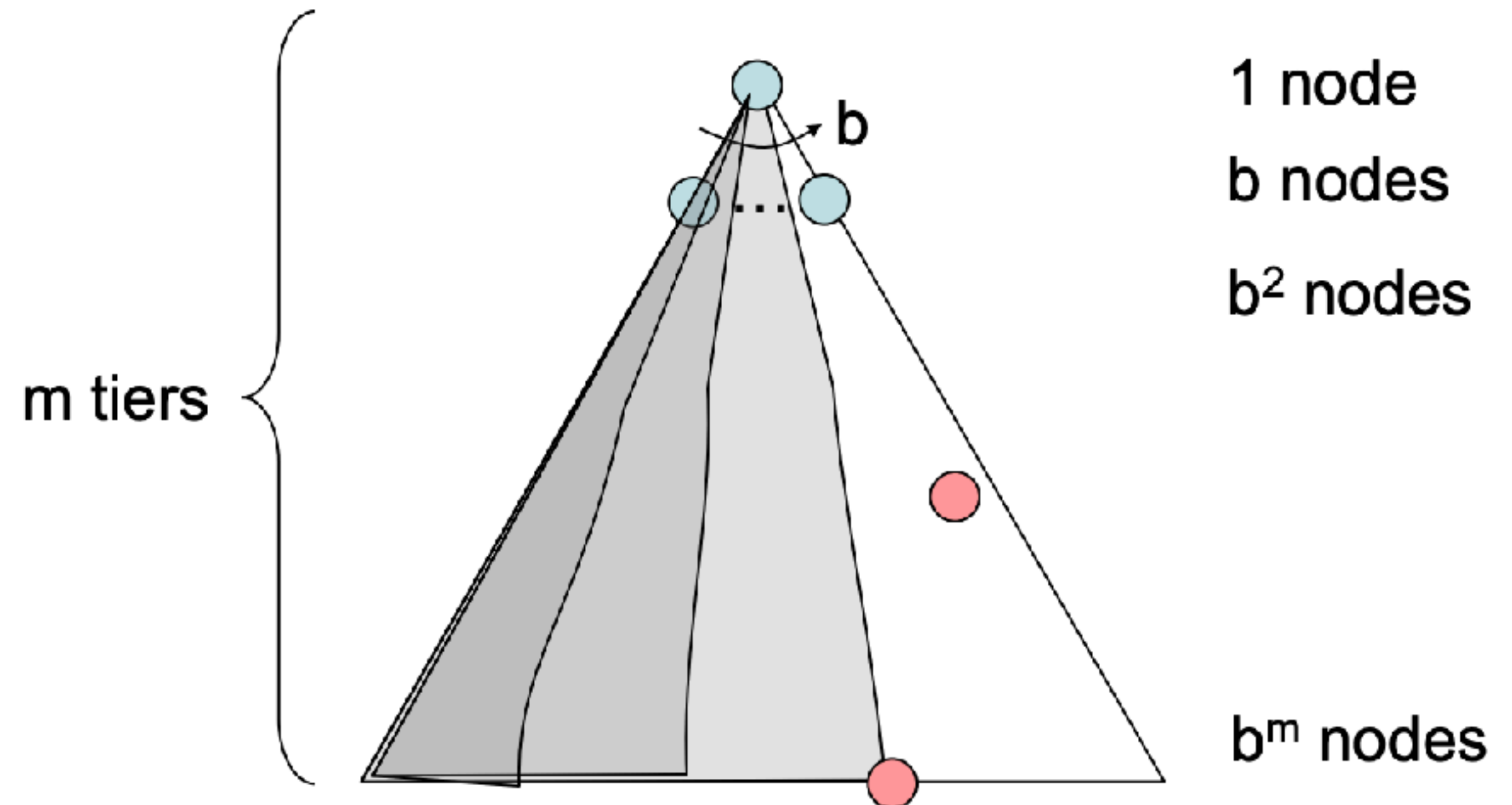
- Only has siblings on path to root, so $\mathcal{O}(bm)$

Is it complete?

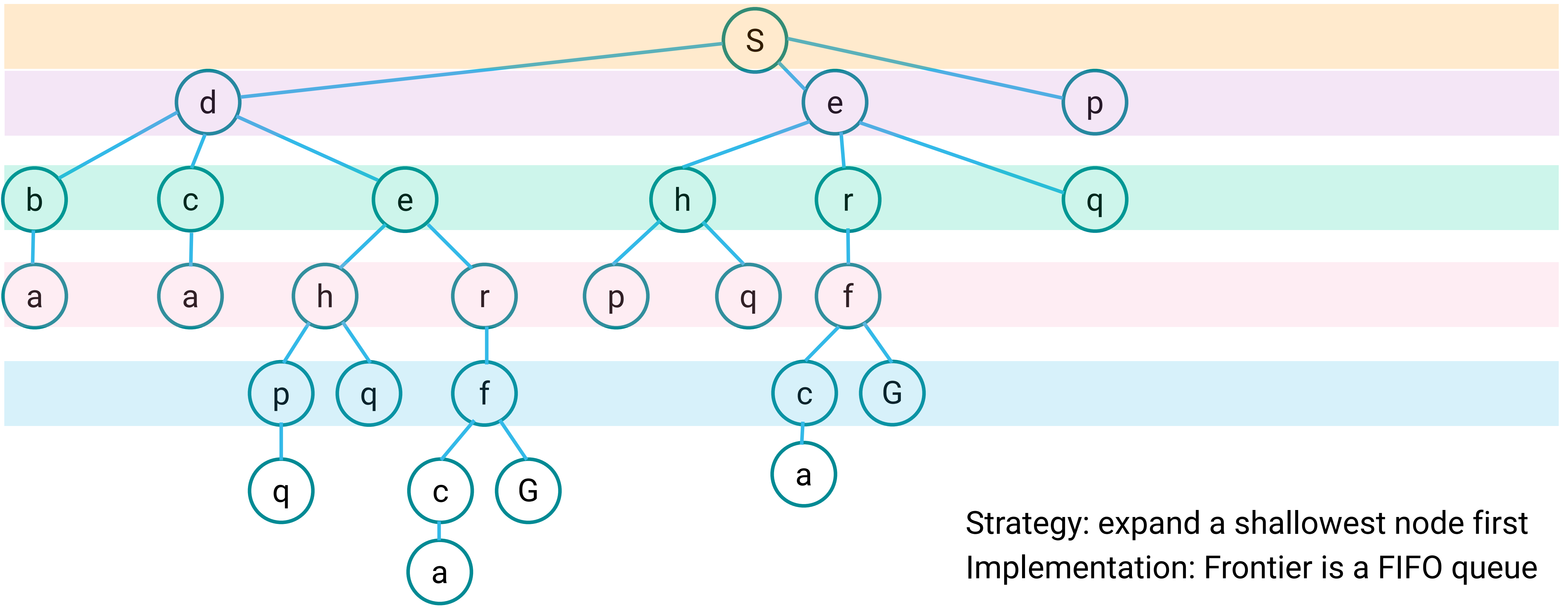
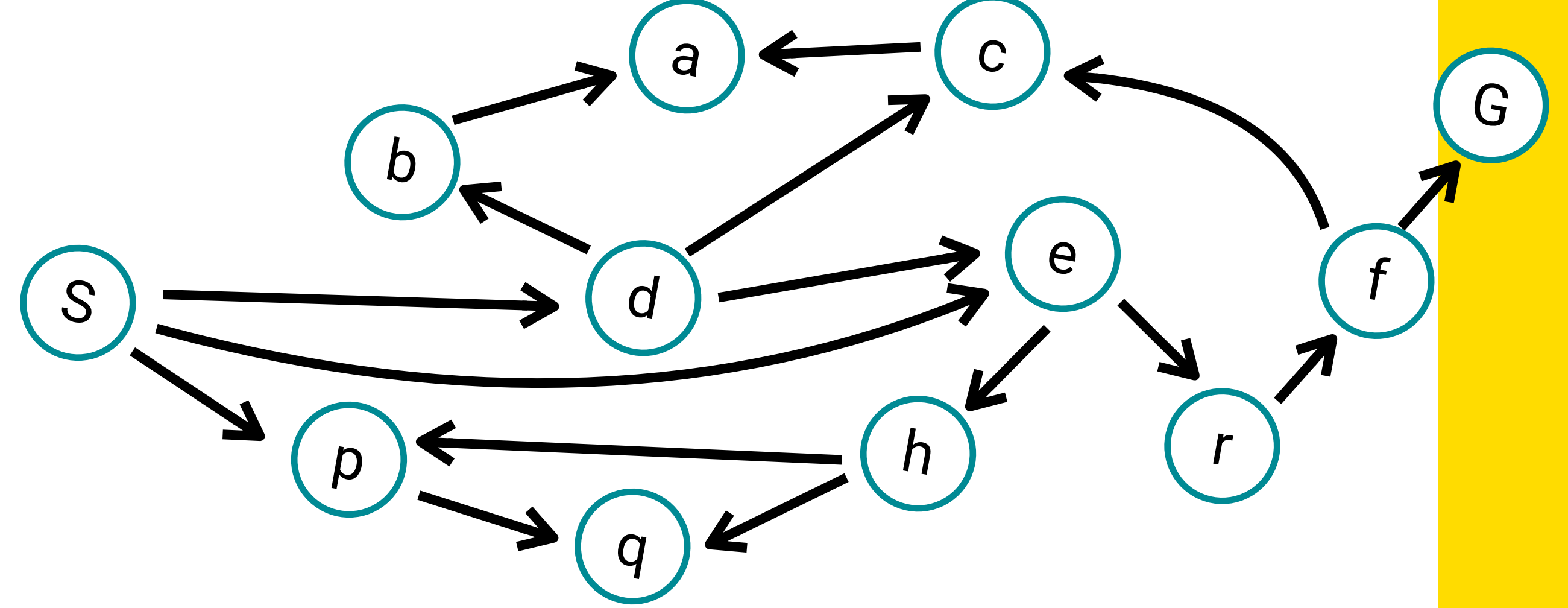
- m could be infinite, so only if we prevent cycles (more later)

Is it optimal?

- No, it finds the “leftmost” solution, regardless of depth or cost



Breadth first search



Strategy: expand a shallowest node first
Implementation: Frontier is a FIFO queue

Properties of Breadth-First-Search (BFS)

What nodes does BFS expand?

- Processes all nodes above shallowest solution
- Let depth of shallowest solution be s
- Search takes time $\mathcal{O}(bs)$

How much space does the frontier take?

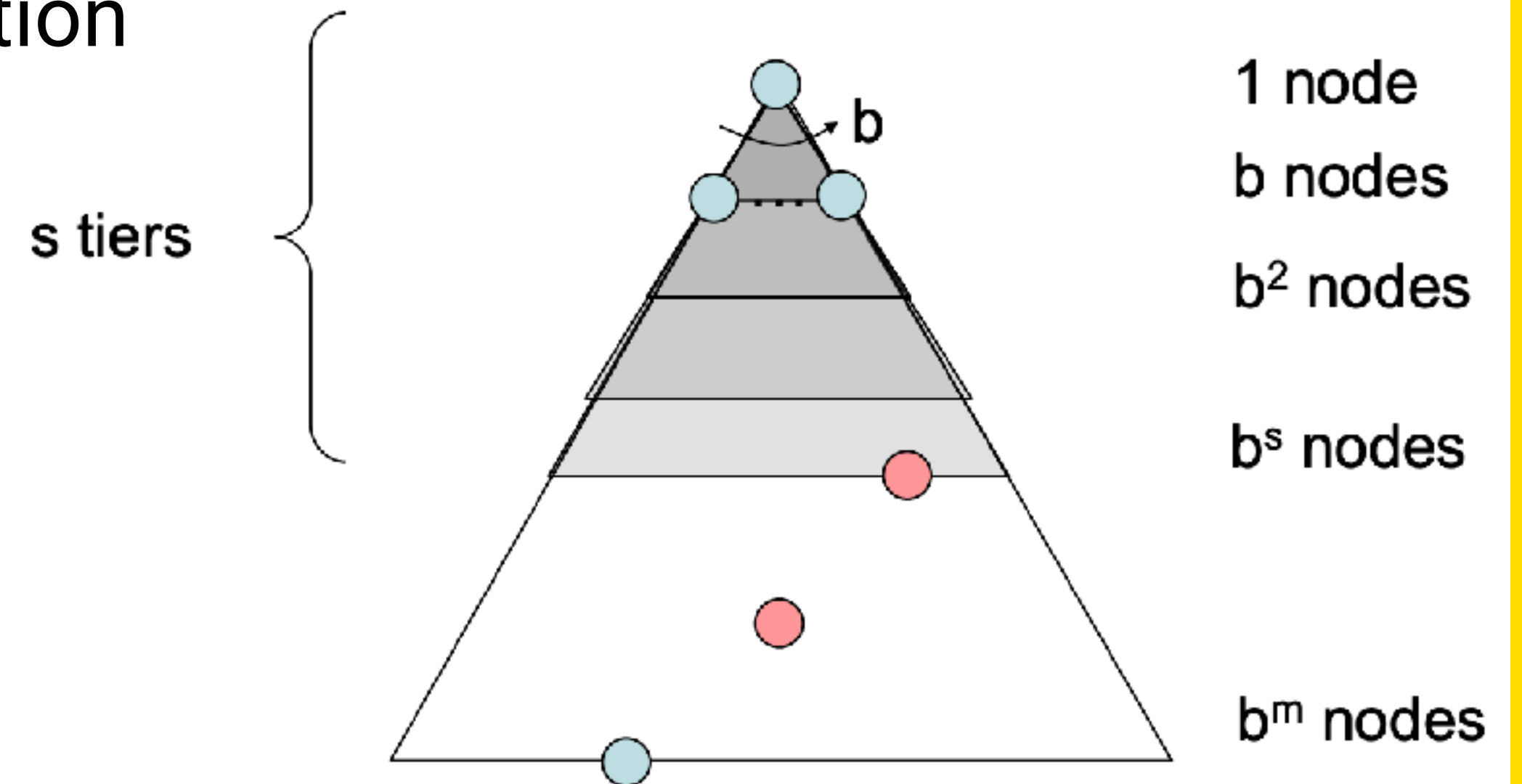
- At end it has roughly the last tier, so $\mathcal{O}(bs)$

Is it complete?

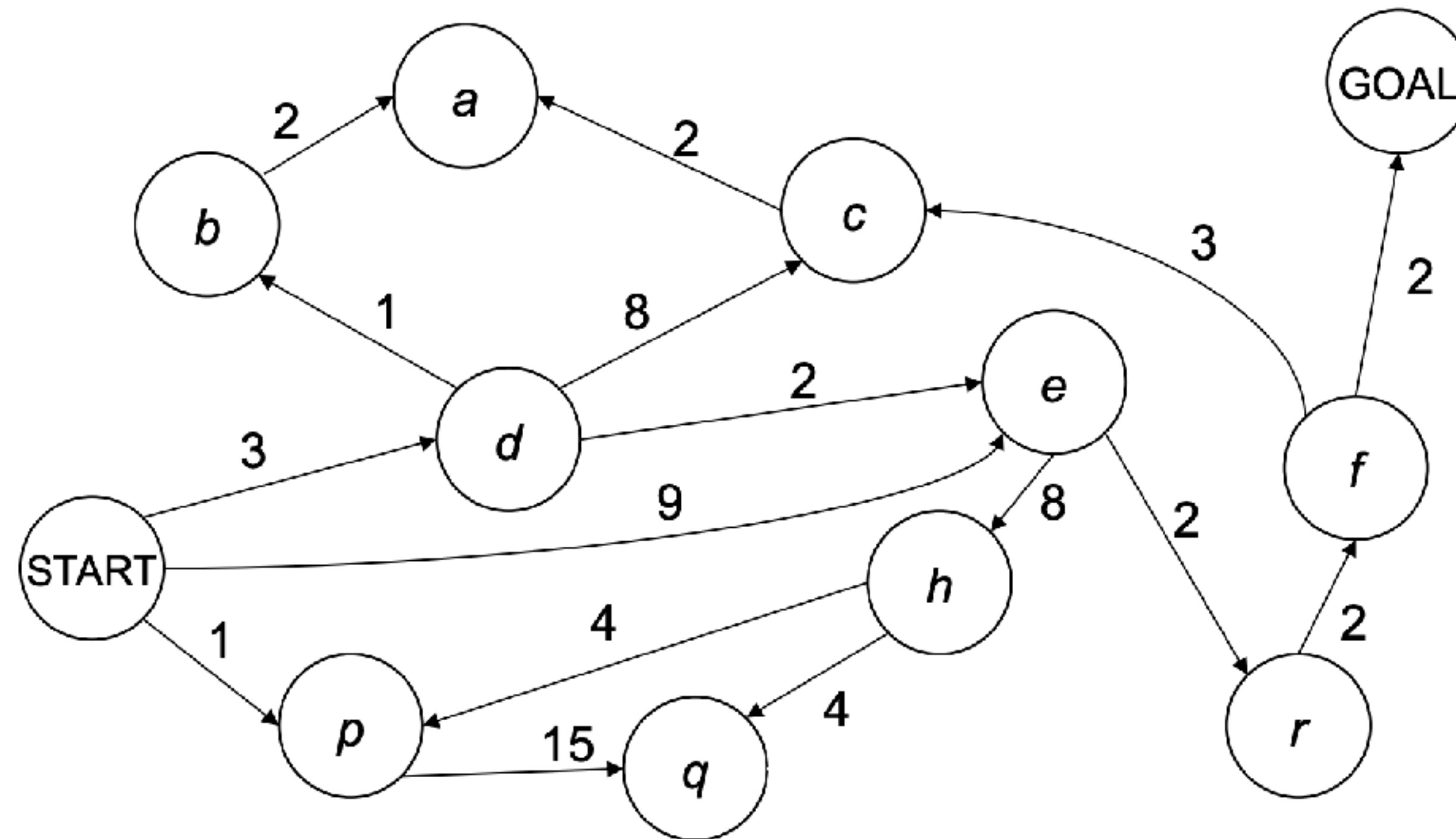
- s must be finite if a solution exists, so yes!

Is it optimal?

- Only if costs are all 1 (more on costs later)



Cost-sensitive search



BFS finds the shortest path in terms of number of actions.

It does *not* find the least-cost path.

We will now cover a similar algorithm which does find the least-cost path.

Frontier (at the start):

- S → p cost 1
- S → d cost 3
- S → e cost 9

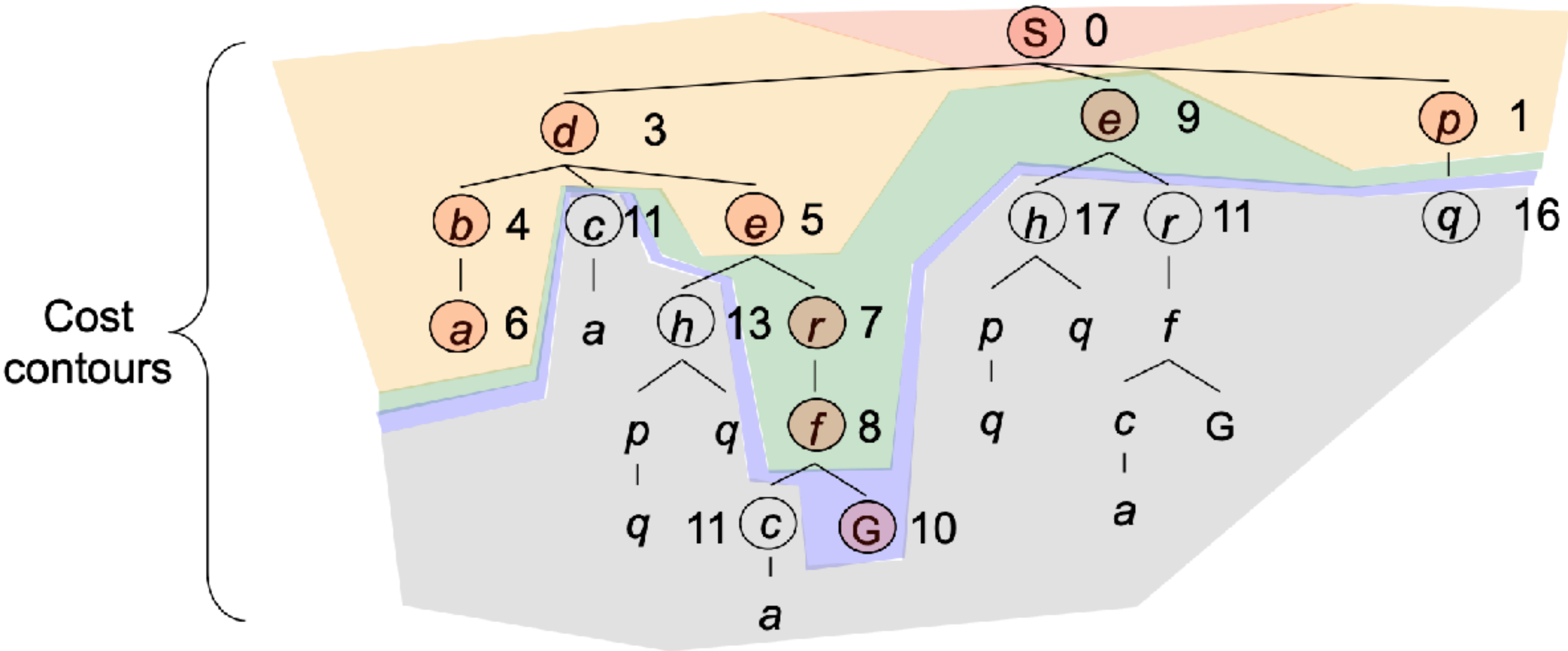
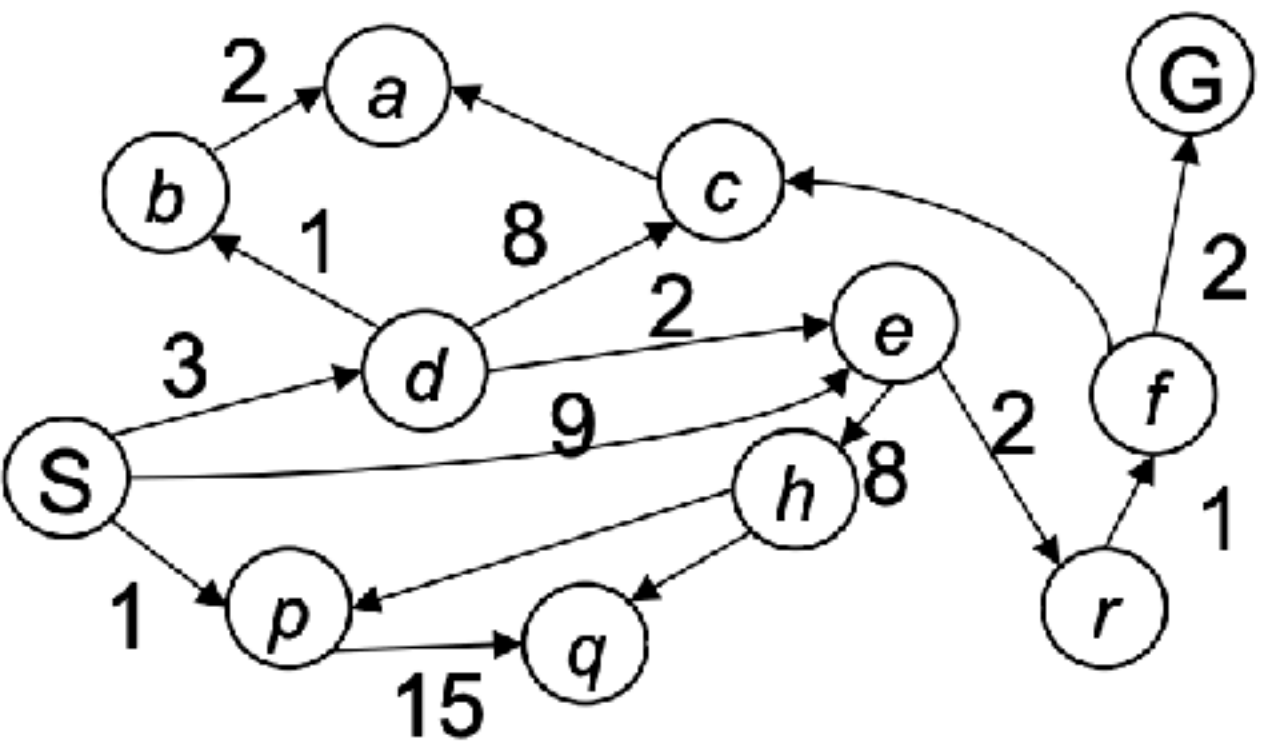
Uniform cost search

Strategy: expand a cheapest node first.

Frontier is a priority queue.

Priority: cumulative cost.

The numbers in the tree are the cumulative cost



Uniform Cost Search (UCS) Properties

What nodes does UCS expand?

- Processes all nodes with cost less than cheapest solution!
- If that solution costs C^* and arcs cost at least e , then the “effective depth” is roughly C^*/e
- Takes time $\mathcal{O}(bC^*/e)$
(exponential in effective depth)

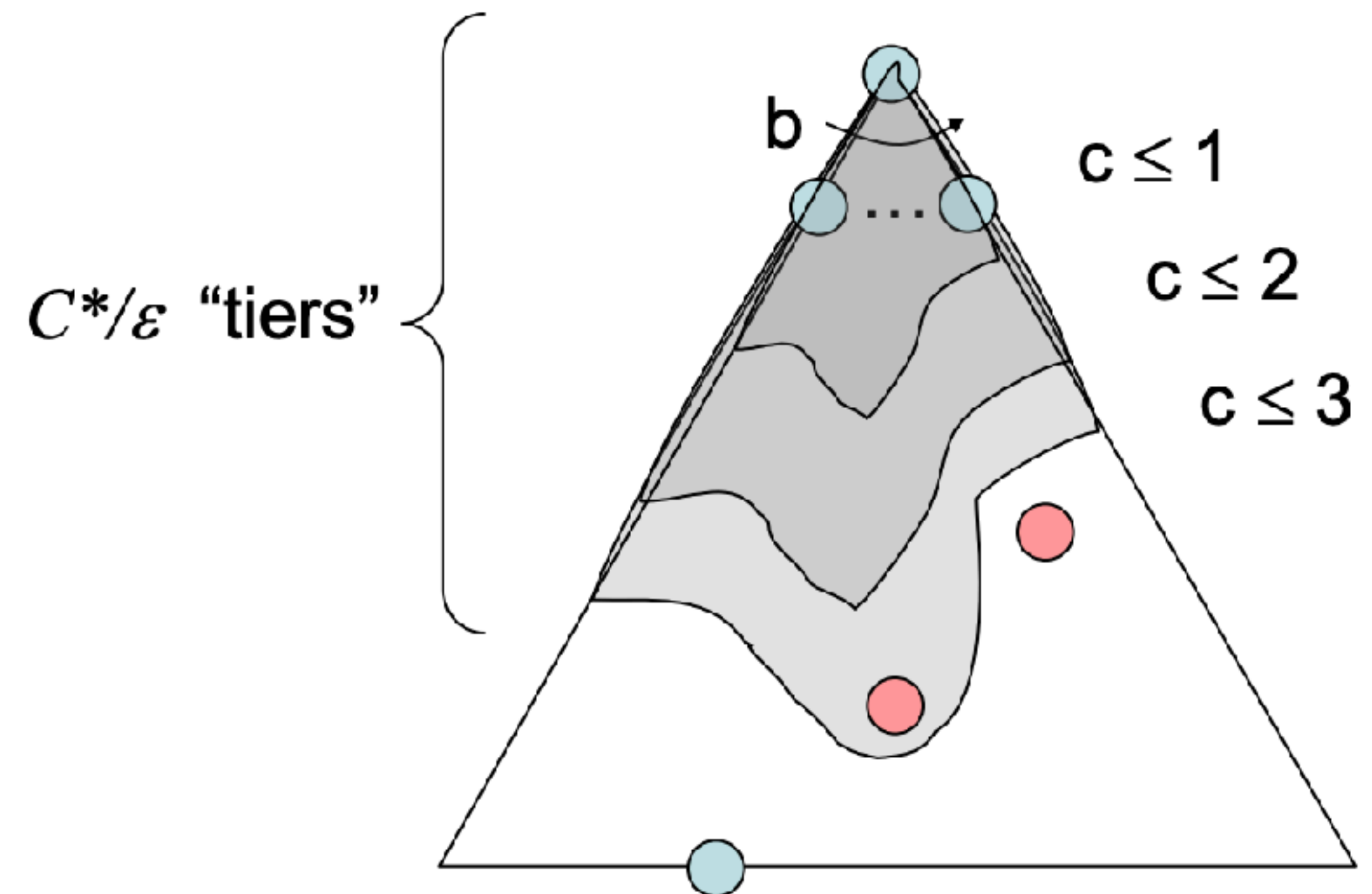
How much space does the fringe take?

- Has roughly the last tier, so $\mathcal{O}(bC^*/e)$

Is it complete?

- Assuming best solution has a finite cost and minimum arc cost is positive, yes!

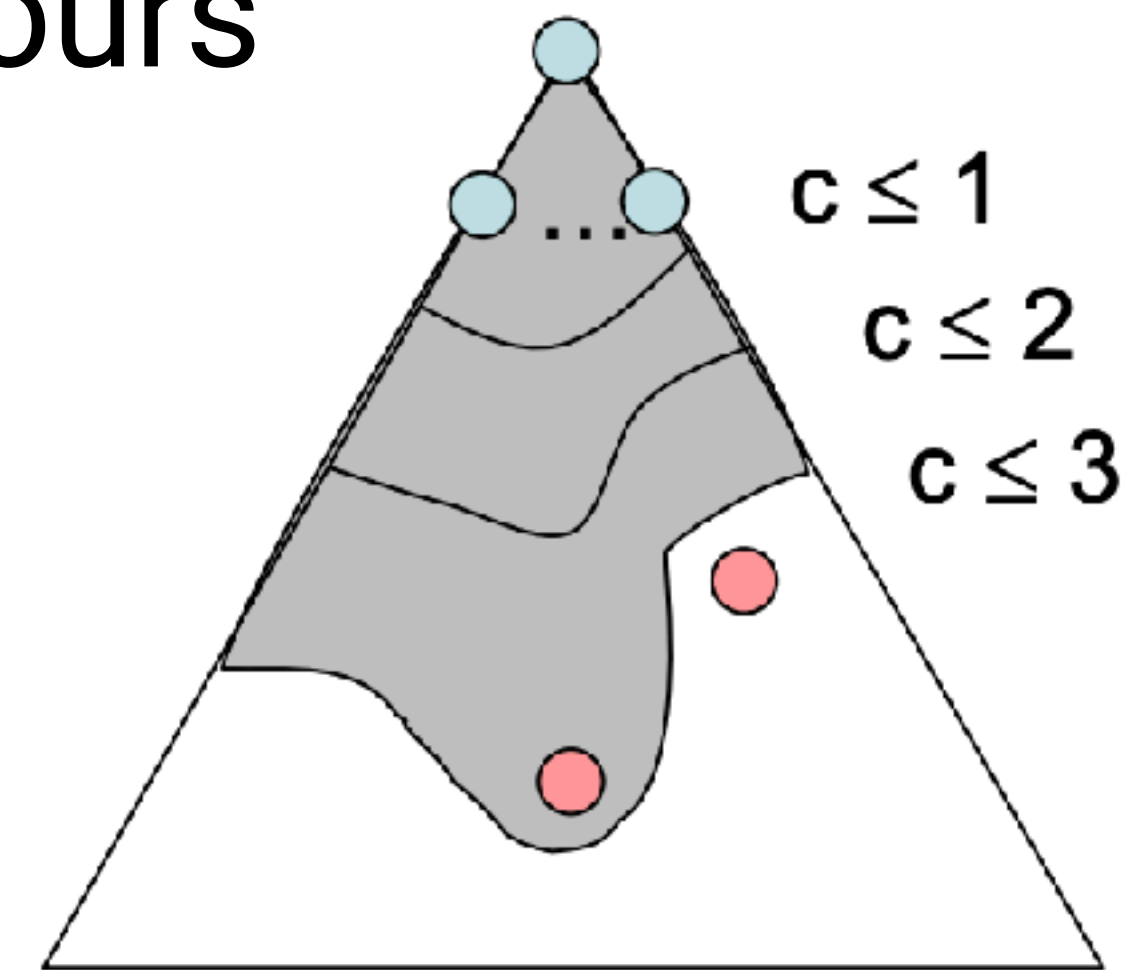
Is it optimal? Yes!



Uniform Cost Search issues

Remember: UCS explores increasing cost contours

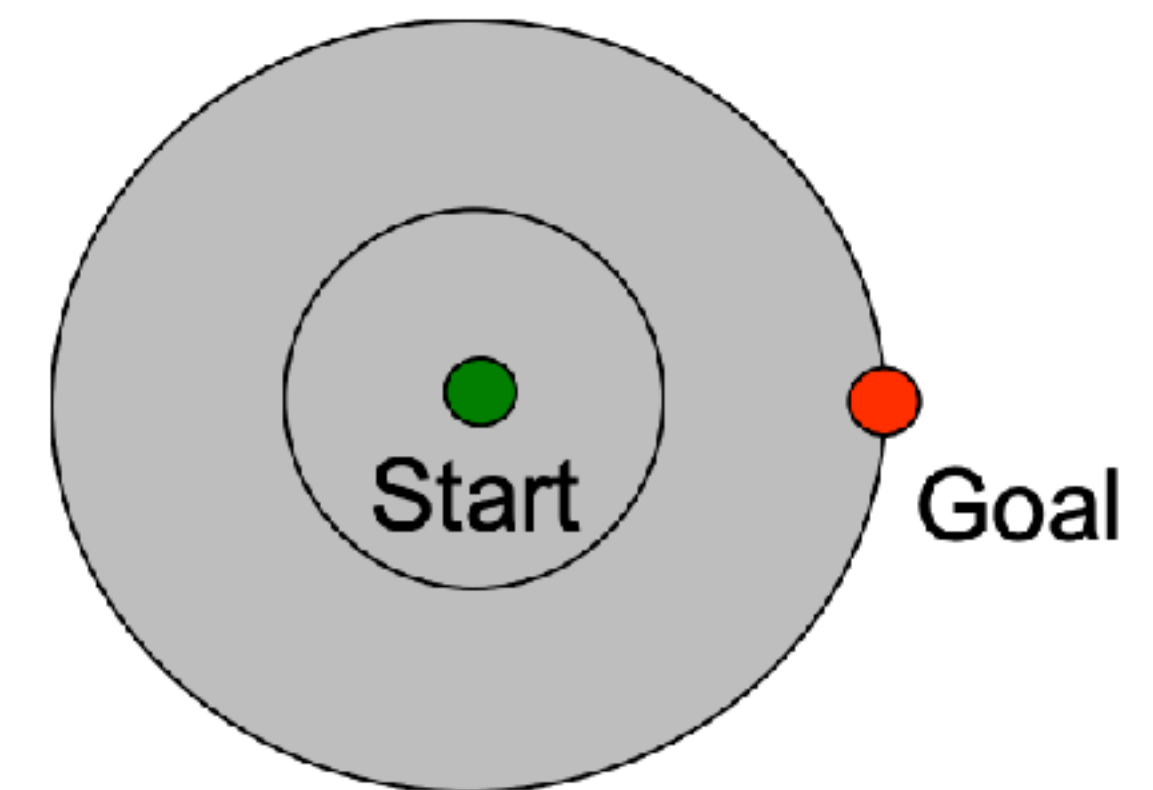
The good: UCS is complete and optimal!



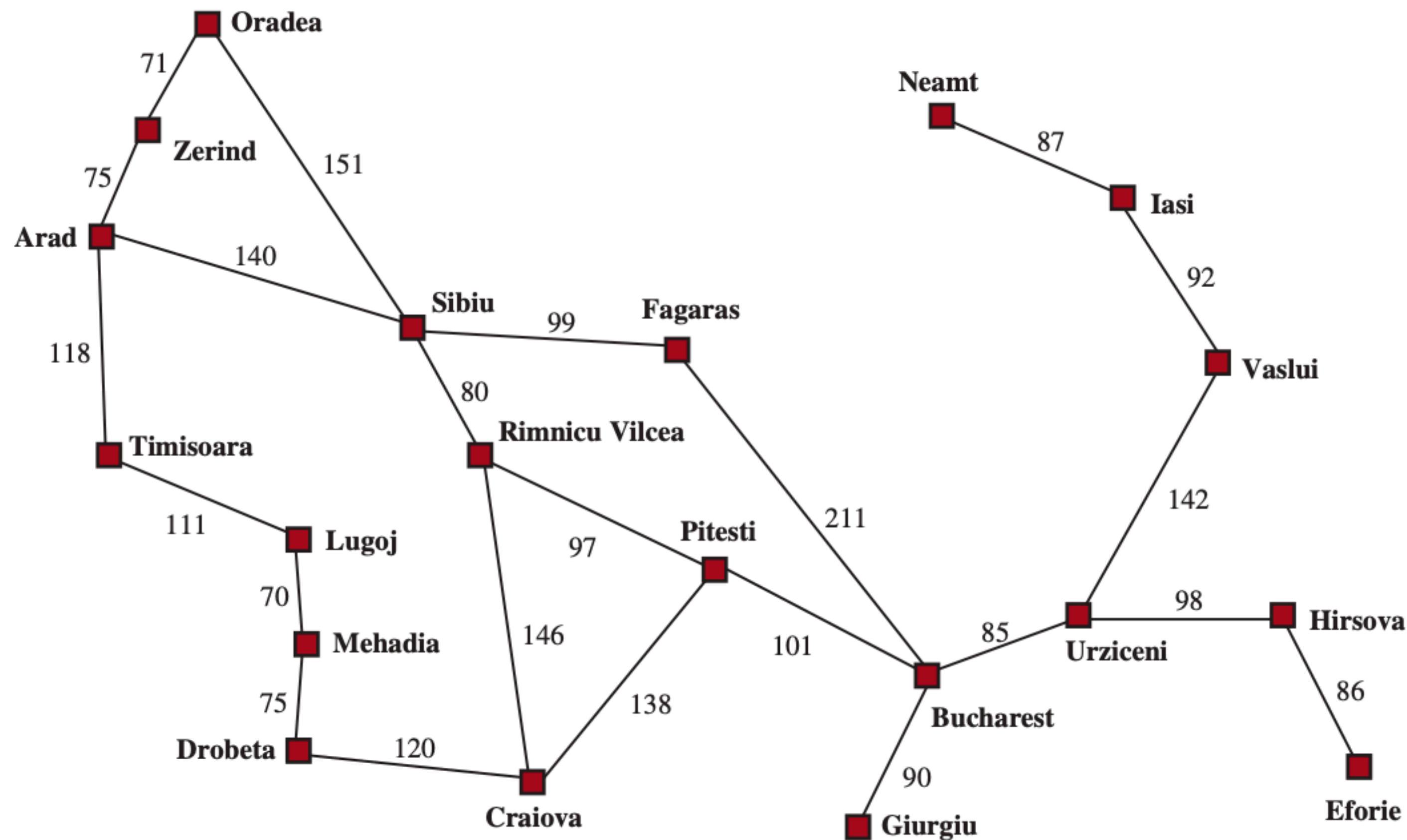
The bad:

Explores options in every “direction”

No information about goal location



Romania revisited, with Heuristics



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

$h(x)$

Greedy search

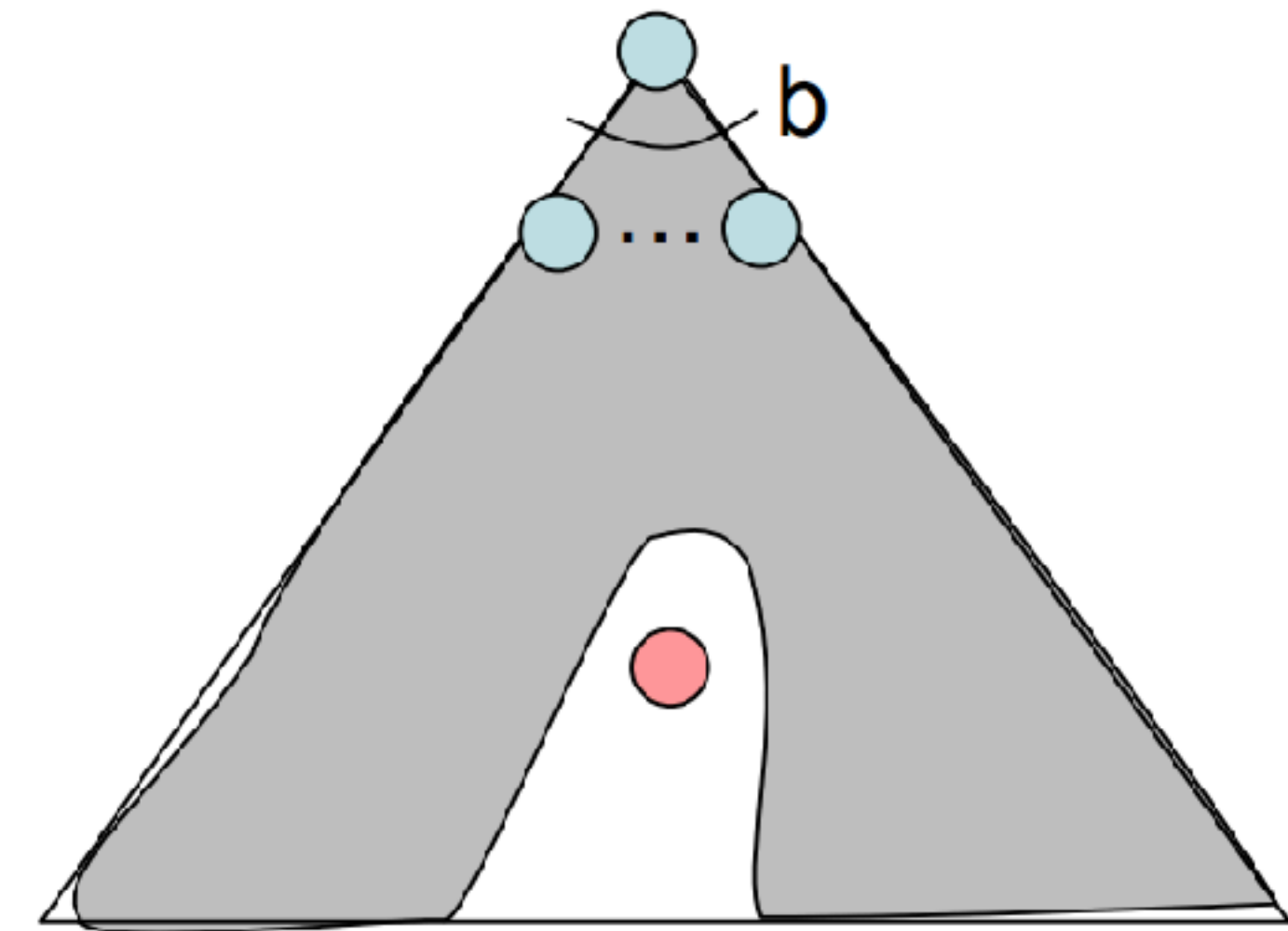
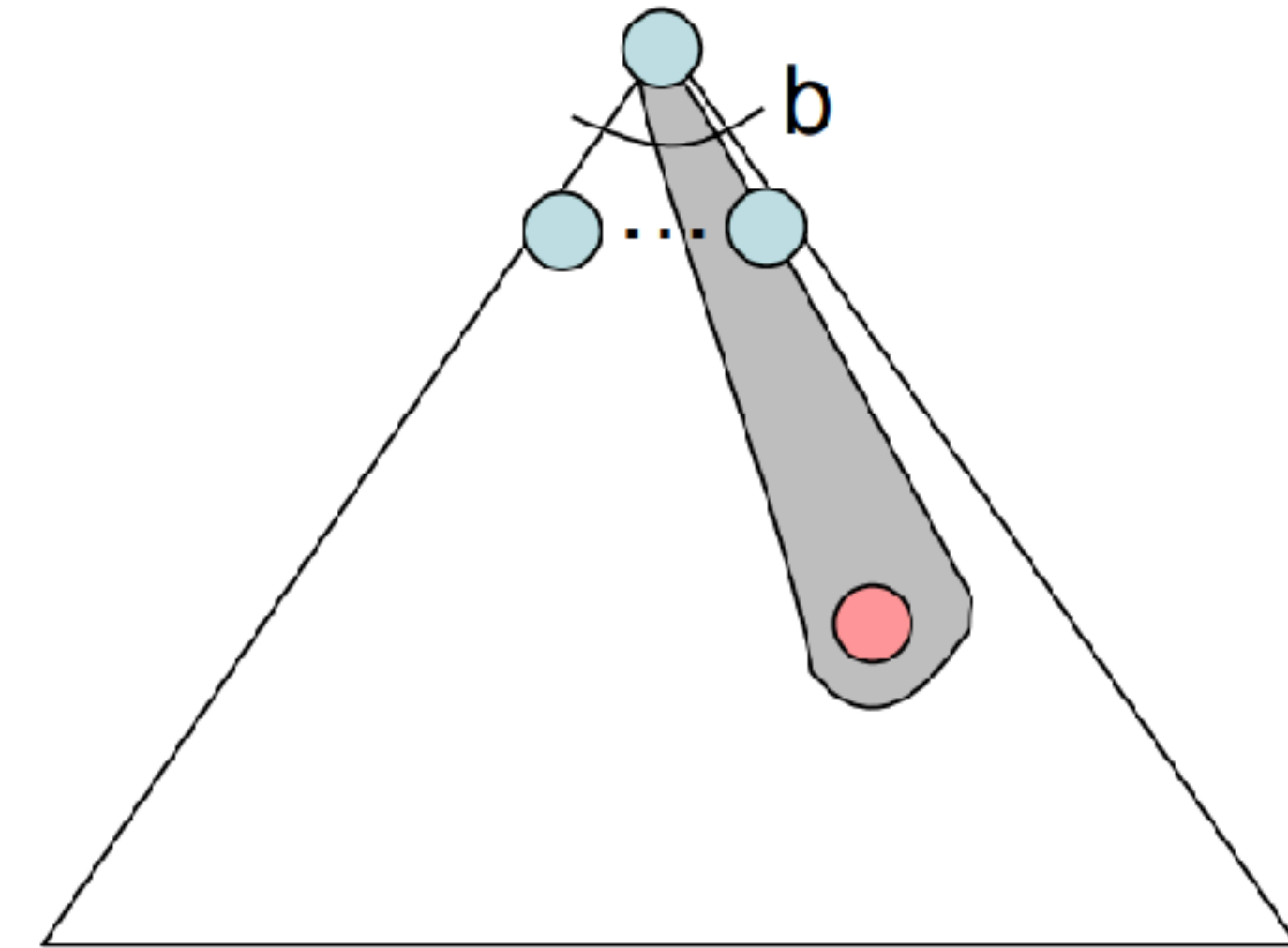
Strategy: expand a node that you think is closest to a goal state

Heuristic: estimate of distance to nearest goal for each state

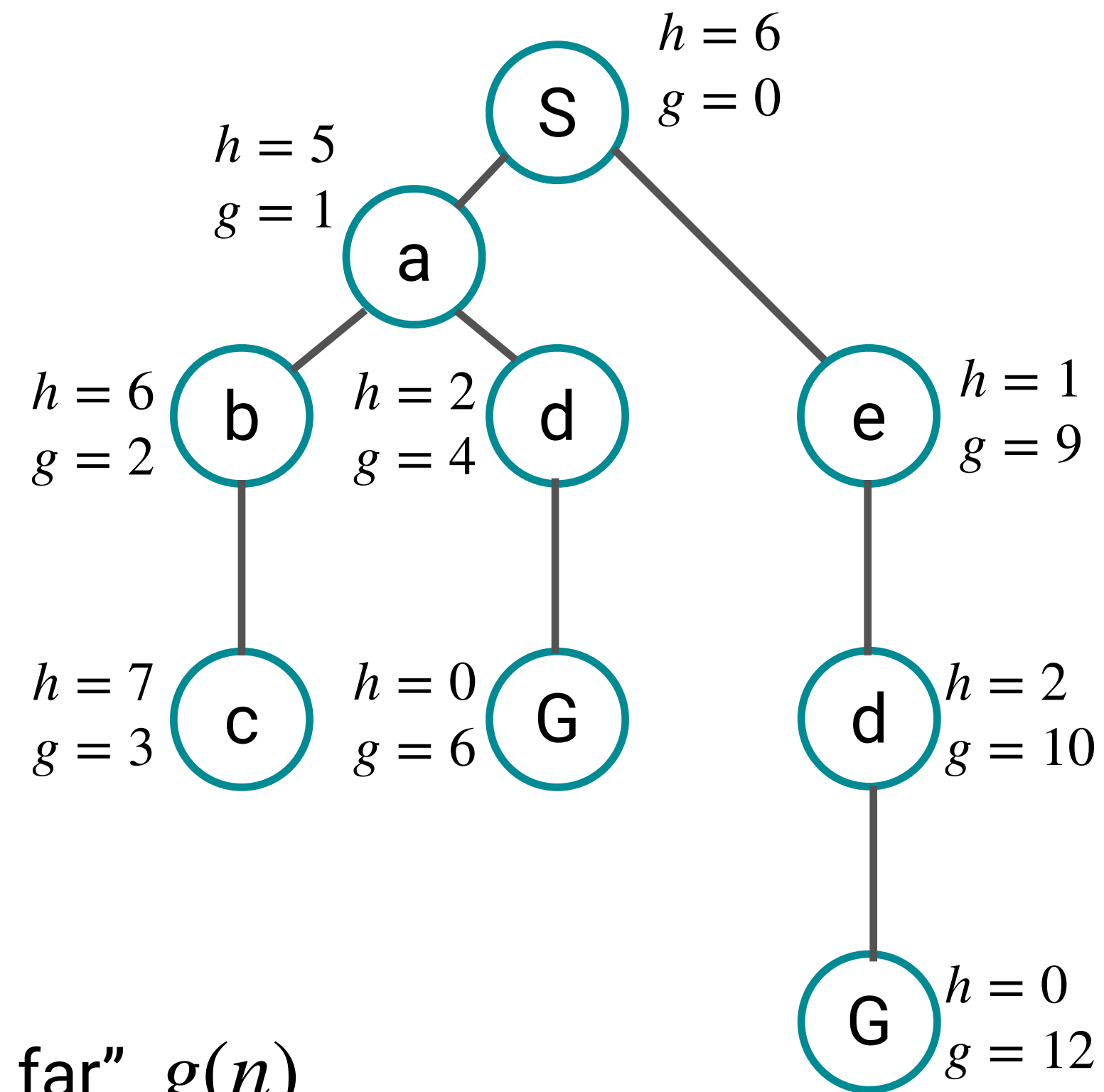
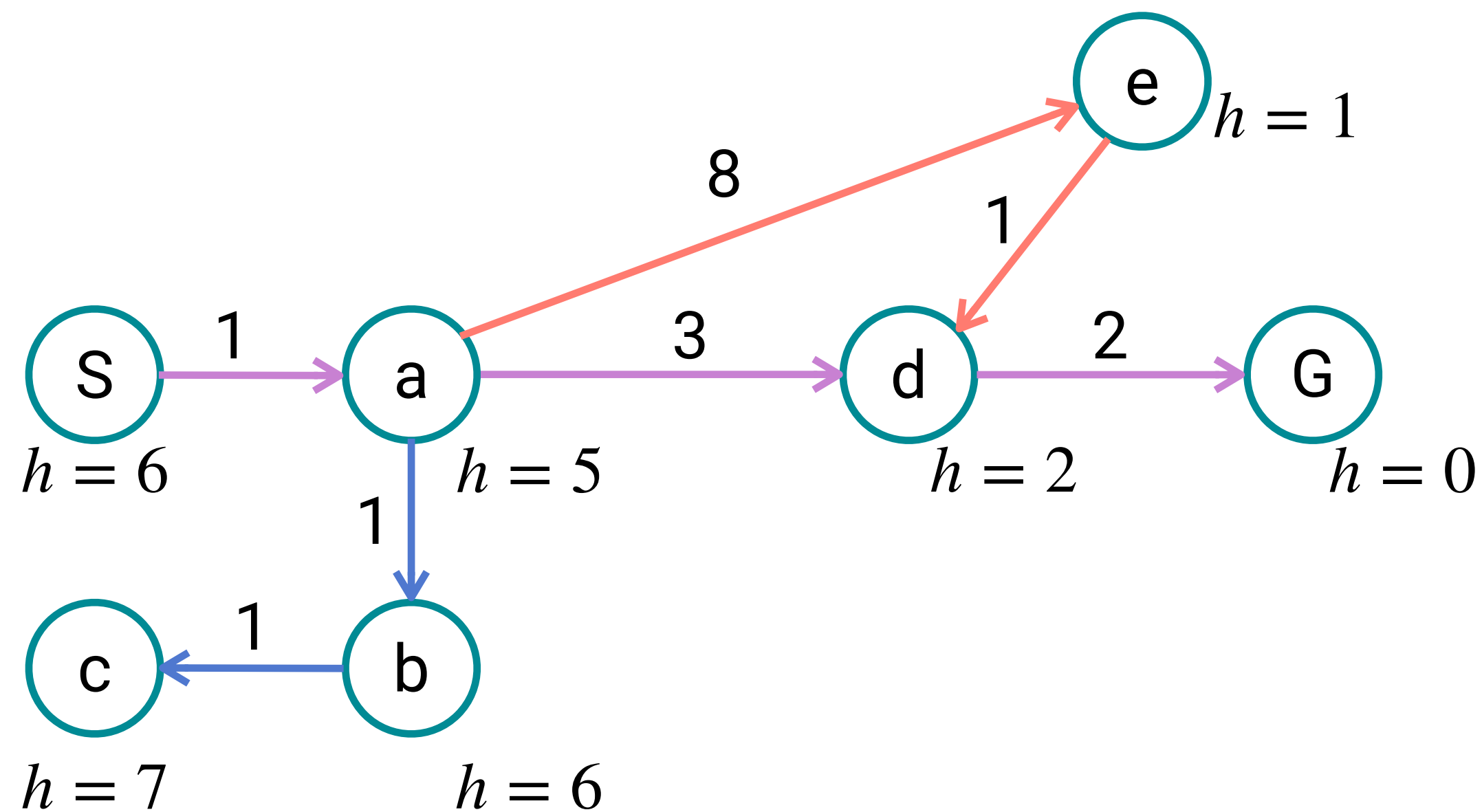
A common case:

- Best-first takes you straight to a (suboptimal) goal

Worst-case: like a badly-guided DFS



Combining greedy and uniform



Uniform-cost orders by path cost, “cost so far” $g(n)$

Greedy orders by goal proximity, “estimated cost to goal” $h(n)$

A* Search orders by the sum, or total estimated cost: $f(n) = g(n) + h(n)$

Heuristics and A*

Does every heuristic work?

For A* optimality, the heuristic must not overestimate the true remaining costs.

A heuristic h is *admissible* (optimistic) if:

$$0 \leq h(n) \leq h^*(n)$$

where $h^*(n)$ is the true cost to the nearest goal.

For example:

The straight-line-distance is an admissible heuristic for finding the route on the map.

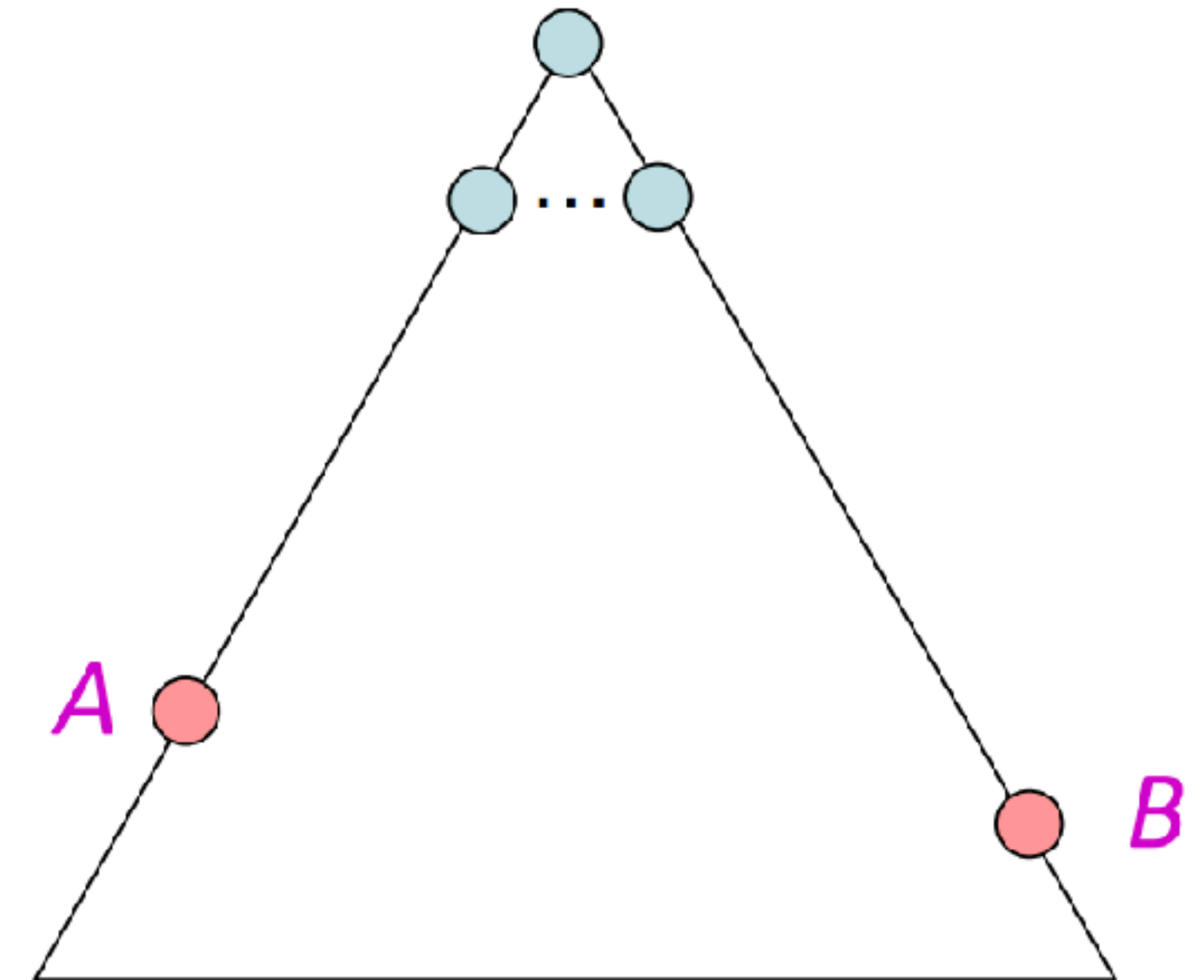
Optimality of A^*

Assume:

- A is an optimal goal node
- B is a suboptimal goal node
- h is admissible

Claim:

- A will be chosen for expansion before B



Optimality of A^*

Proof:

Imagine B is on the frontier

Some ancestor n of A is on the frontier, too (maybe A itself)

Claim: n will be expanded before B

1. $f(n) \leq f(A)$

$$f(n) = g(n) + h(n)$$

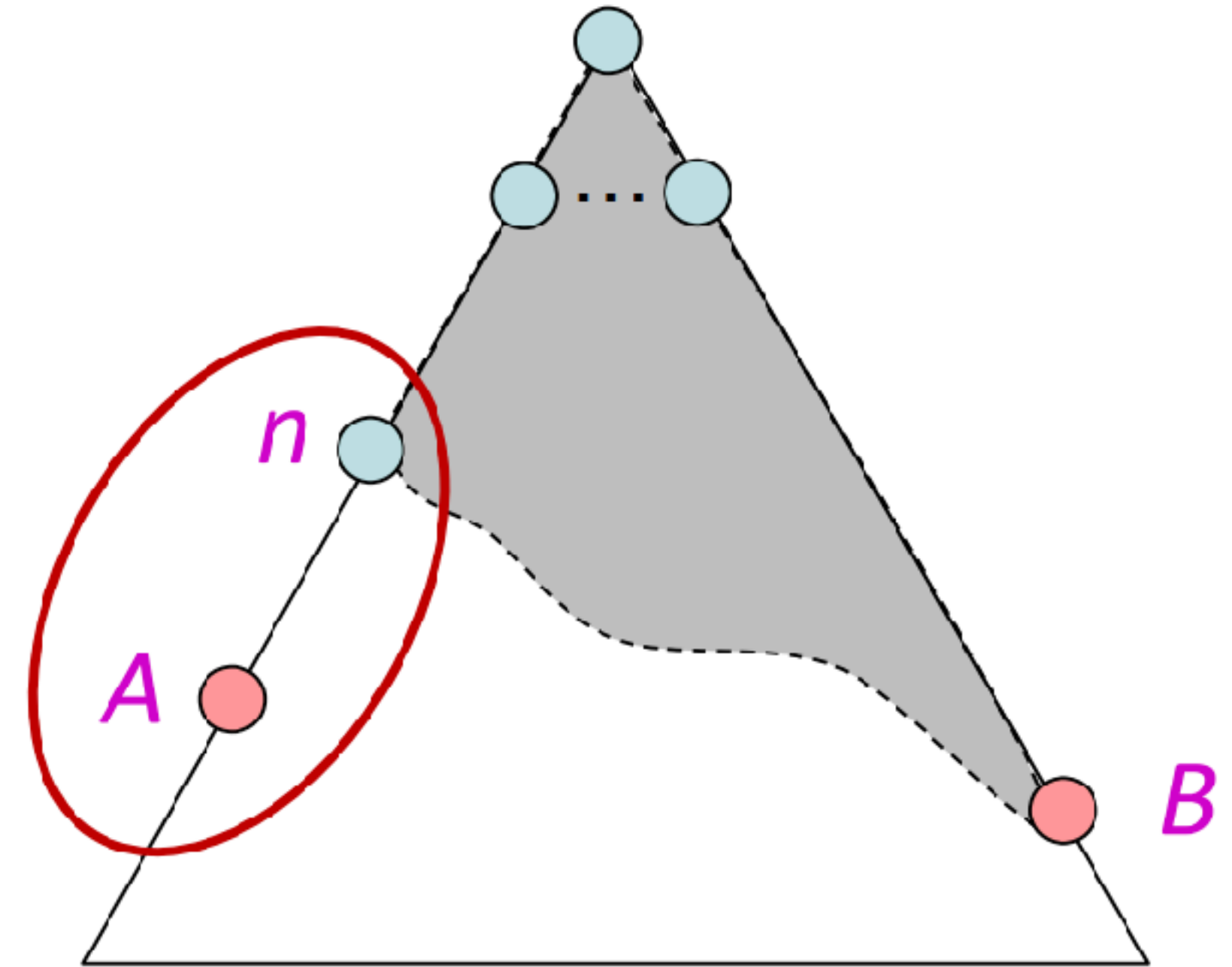
$$f(n) \leq f(A)$$

$$g(A) = f(A)$$

definition of f -cost

Admissibility of h

$h = 0$ at the goal



Optimality of A^*

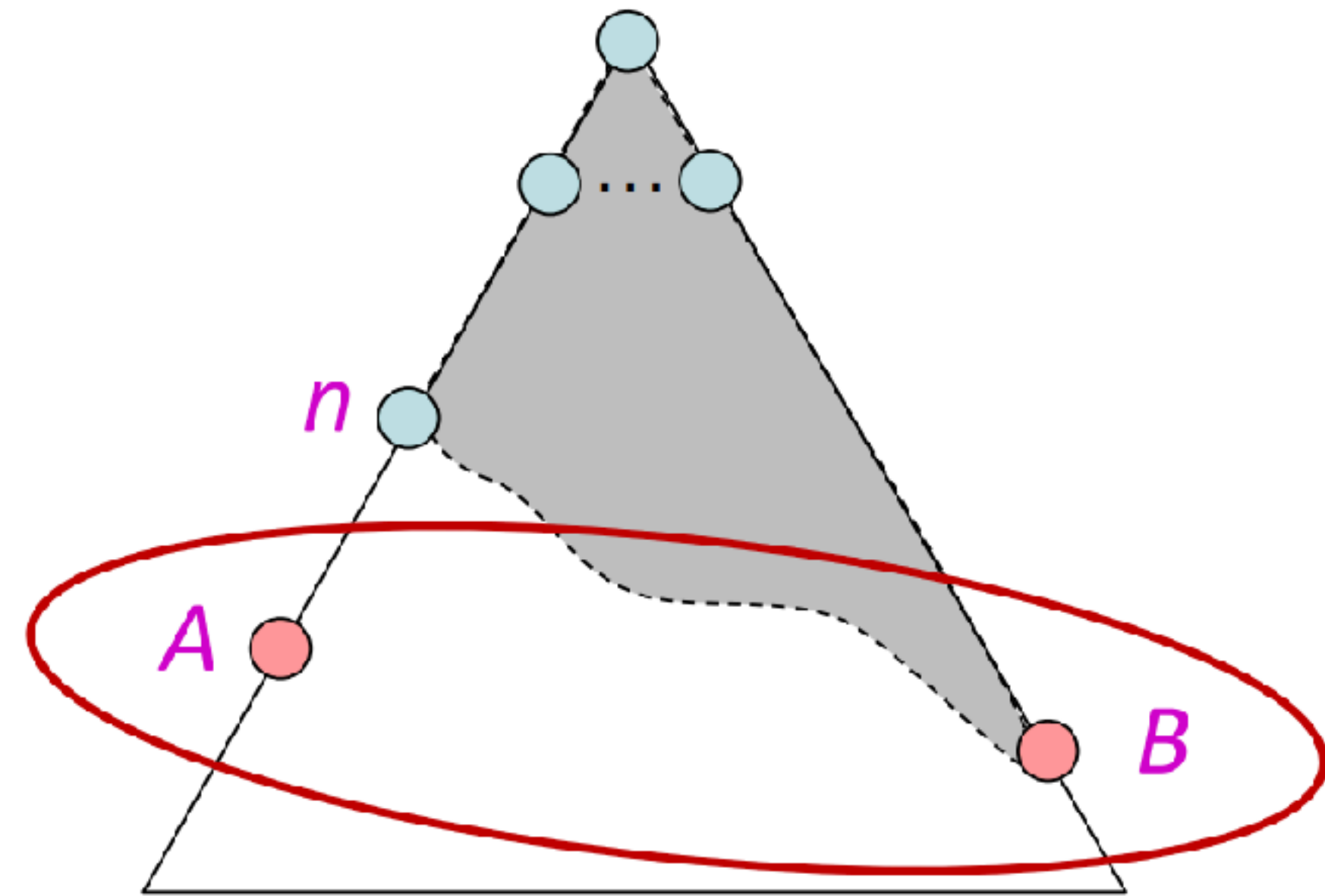
Proof:

Imagine B is on the frontier

Some ancestor n of A is on the frontier, too (maybe A itself)

Claim: n will be expanded before B

1. $f(n) \leq f(A)$
2. $f(A) < f(B)$



$$g(A) < g(B)$$

Sub-optimality of B

$$f(A) < f(B)$$

$h = 0$ at the goal

Optimality of A^*

Proof:

Imagine B is on the frontier

Some ancestor n of A is on the frontier, too (maybe A itself!)

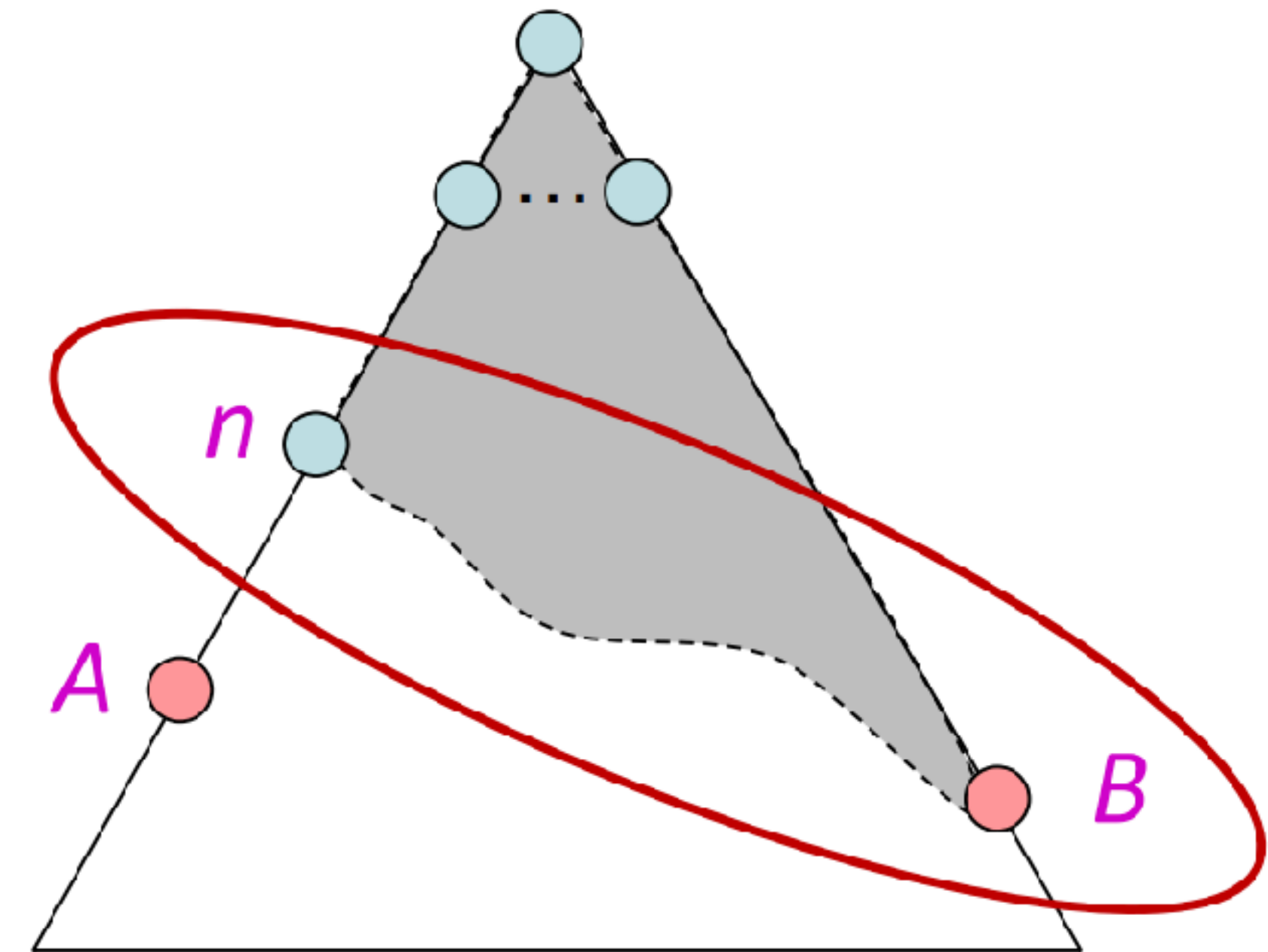
Claim: n will be expanded before B

1. $f(n) \leq f(A)$
2. $f(A) < f(B)$
3. n is expanded before B

All ancestors of A are expanded before B

A is expanded before B

A^* tree search is optimal



$$f(n) \leq f(A) < f(B)$$

From search to planning

Today's problem was deliberately simple:

- (Robot) state: location
- Actions: move to the next place
- Goal: reach the destination
- Cost: number of moves, or distance

This is enough for navigation.

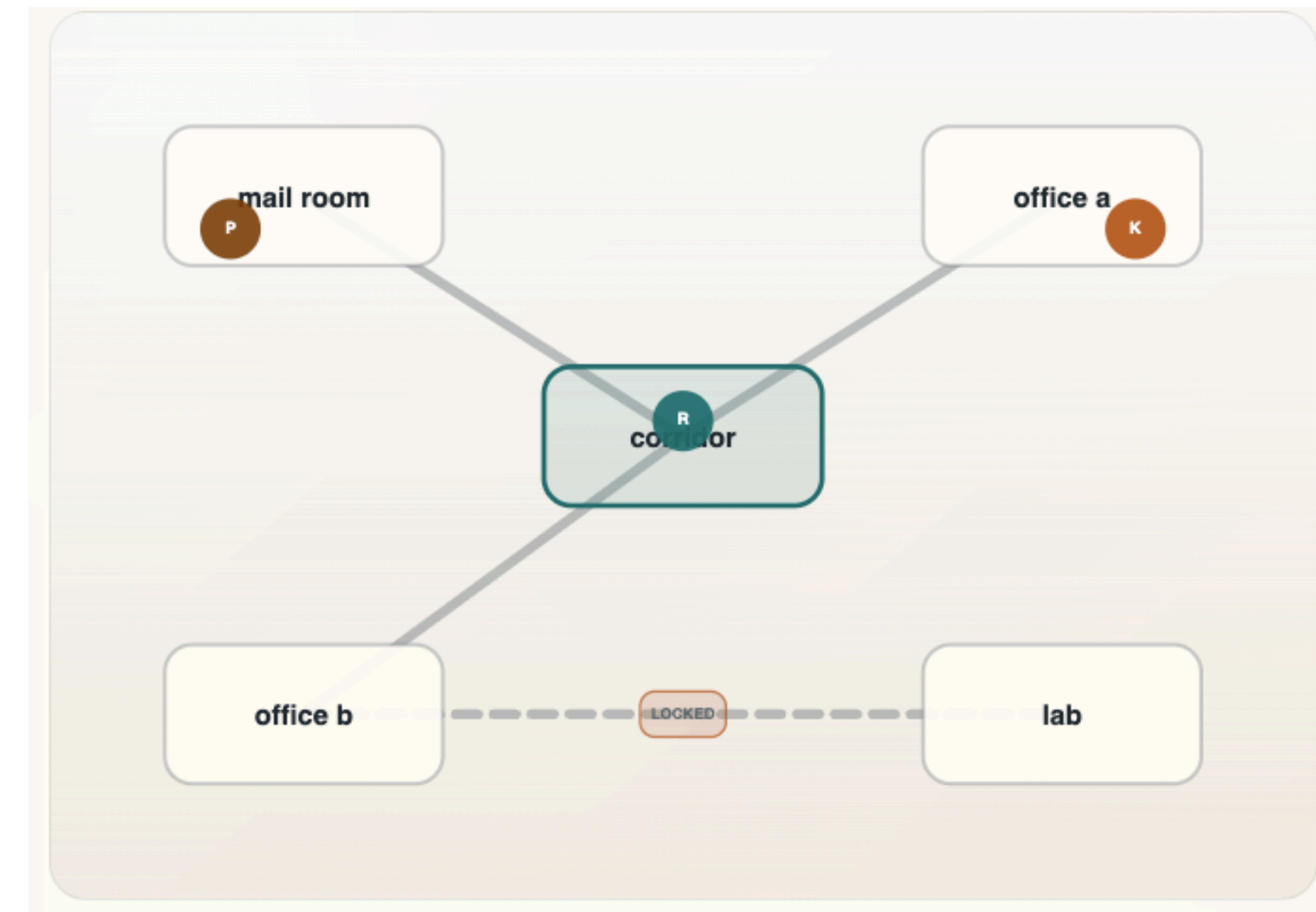
Many problems are not just that.

What if the robot delivers something?

Example:

- Parcel in one room
- Door is locked
- Key card is in another room
- Robot has to carry parcel to another room

Now, finding a path to the goal is not enough.



Planning - search over action sequences

This week: search

- States are locations
- Actions are movements
- Goal is the destination

Next week: planning

- States are facts about the world
- Actions have preconditions and effects
- Goal is a set of conditions

How do we represent actions and search for a valid plan?

What you should know after this week

- what is an agent (in AI)?
- how do we formulate a problem for search?
- what is the difference between a state space and a search tree?
- how do DFS, BFS, and uniform-cost differ?
- what is a heuristic?
- what is the intuition behind A^* ?